



Università Politecnica delle Marche

A.A. 2015/2016

File System

Luca Spalazzi

spalazzi@dii.univpm.it

www.univpm.it/luca.spalazzi



File-System

■ Interfaccia del File System

- Tipo di Dato Astratto File
- File Ordinari
- Directory
- File Speciali
- Protezioni

■ Realizzazione del File System

- Struttura dei file system
- Realizzazione delle directory
- Realizzazione dei file (assegnazione dei blocchi liberi)
- Cache (dei blocchi e delle pagine) e Buffering

■ File System di Linux

- Virtual File System
- Ext2 File System
- Ext3 File System
- Proc File System
- Network File System



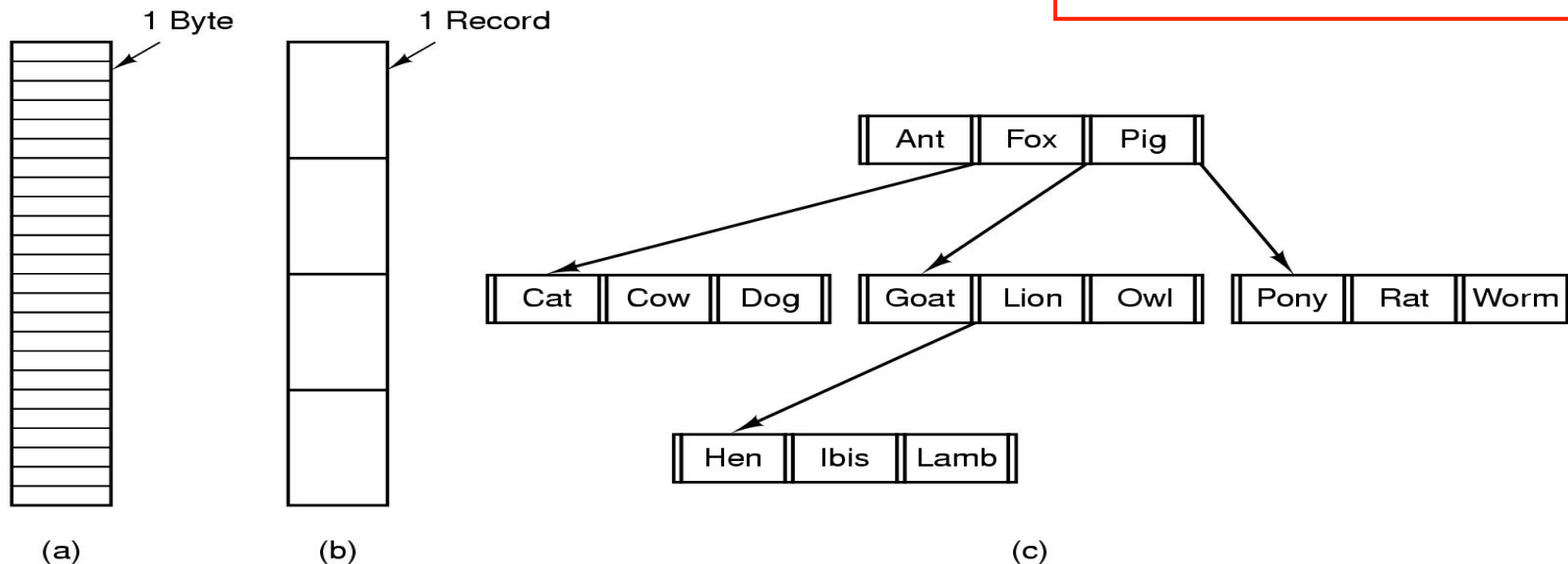
File

- Un file è un **tipo di dato astratto**.
- Definire un file significa quindi definire:
 - Struttura
 - Attributi
 - Operazioni
- Fornisce agli utenti (e in particolare ai programmatori) una interfaccia uniforme alla gestione dei dispositivi di I/O
- In particolare il concetto di file è utile per la gestione delle memorie di massa. In questo caso, ogni file denota uno **spazio logico degli indirizzi contiguo**, indipendentemente da come è realizzato.

Struttura dei File

- a) Assenza di struttura – sequenza di byte
- b) Struttura a Record – sequenza di record
 - Record di lunghezza fissa
 - Record di lunghezza variabile
- c) Struttura Complessa
 - Grafo/Albero di record
 - Documento formattato (es. Ipertesti)

Le strutture (b) e (c) si possono realizzare con struttura (a) (sequenze di byte) dotata di opportuni caratteri di controllo



■ Chi gestisce la struttura del file?

- Sistema operativo
- Programma



Attributi dei File

- **Nome** – è l'informazione che permette agli **utenti** di identificare in modo univoco il file.
- **Identificatore** - è l'informazione che permette al **SO** di identificare in modo univoco il file.
- **Tipo** – questa informazione serve ai sistemi che gestiscono file di tipo diverso.
- **Locazione** – è il puntatore al dispositivo e alla locazione del file in tale dispositivo.
- **Dimensione** – è l'attuale dimensione del file.
- **Protezione** – sono informazioni di controllo degli accessi al file (chi può leggere, scrivere, far eseguire il file).
- **Ora, data, e identificatore dell'utente** – sono informazioni relative alla creazione, all'ultima modifica e all'ultimo uso del file. Servono per la protezione ed il controllo dell'uso.

Gli attributi dei file sono conservati nella struttura directory, che risiede a sua volta nella memoria secondaria.



Operazioni sui File

- **Create** – per creare un file bisogna compiere due passi:
 - 1) si deve trovare lo spazio per il file nel file system;
 - 2) si deve creare un nuovo elemento nella directory dove registrare i suoi attributi.
- **Open** – questa operazione riceve il nome del file, lo cerca nella directory, e copia l'elemento ad esso associato in memoria (nella **tabella dei file aperti**). In alcuni SO l'apertura dei file avviene implicitamente la prima volta che il processo fa riferimento al file.
- **Write** – il file system deve mantenere un **puntatore di scrittura** contenente la locazione nel file in cui deve avvenire la prossima operazione di scrittura. Il file deve essere già aperto.
- **Read** – il file system deve mantenere un **puntatore di lettura** contenente la locazione nel file in cui deve avvenire la prossima operazione di lettura. Il file deve essere già aperto.
- **Seek** – spesso il puntatore di lettura e quello di scrittura coincidono, la seek cambia il valore a tale puntatore (senza operazioni di I/O).
- **Delete** – si cerca l'elemento relativo al file nella directory, si rilascia lo spazio assegnato al file e si elimina l'elemento di directory.
- **Truncate** – si cancella il contenuto del file mantenendo i suoi attributi.
- **Close** – copia l'elemento contenuto nella tabella dei file aperti nella directory. In alcuni SO la chiusura avviene implicitamente quando il processo termina.



Tipi di File – Estensione dei Nomi

file type	usual extension	function
executable	exe, com, bin or none	read to run machine- language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rrf, doc	various word-processor formats
library	lib, a, so, dll, mpeg, mov, rm	libraries of routines for programmers
print or view	ps, pdf	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes com- pressed, for archiving or storage
multimedia	mpeg, mov, rm	binary file containing audio or A/V information



Metodi di Accesso

Metodi Base

■ Accesso Sequenziale

read next legge il record nella posizione corrente (**cp**)

write next scrive il record nella posizione corrente, quel record diventa l'ultimo del file

reset riporta la posizione corrente all'inizio del file (**cp = 0**)

- non è possibile fare una *read* dopo una *write* se prima non si fa una *reset*
- non è possibile fare una *rewrite*

■ Accesso Diretto

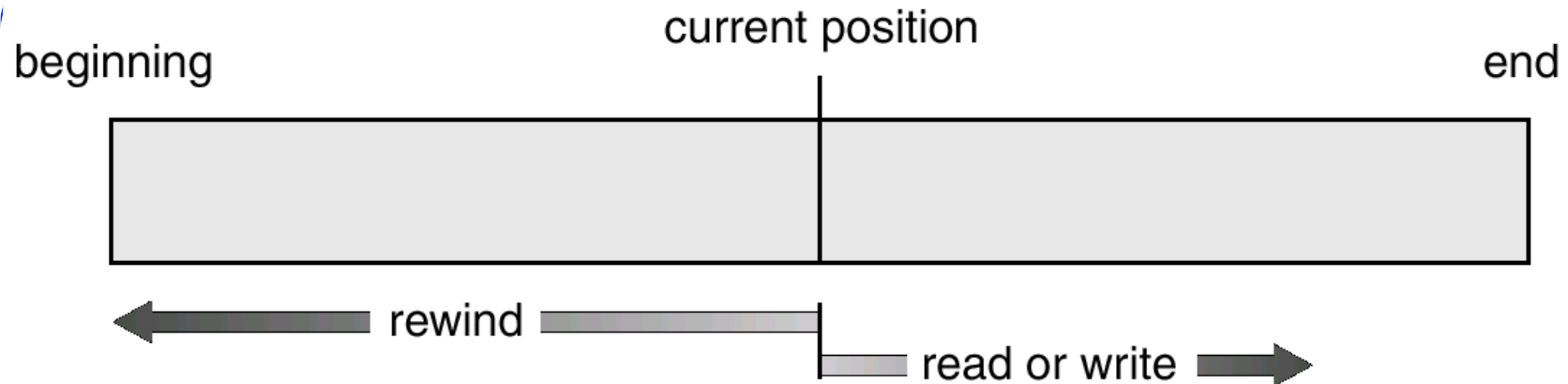
read n legge il blocco n-esimo (**cp = n**)

write n scrive il blocco n-esimo (**cp = n**)

rewrite n riscrive il blocco n-esimo

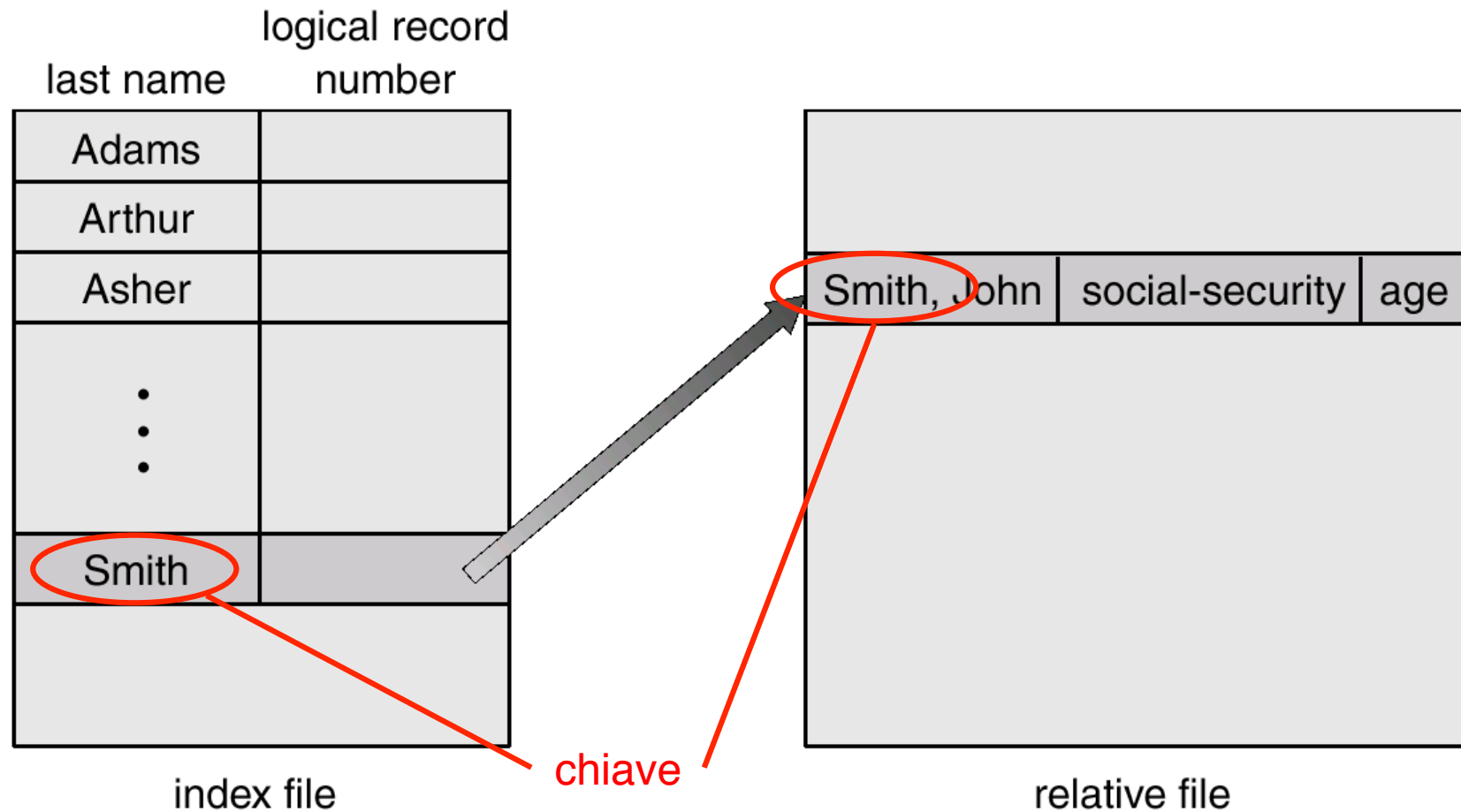
- è la base di altri metodi di accesso: **accesso con indice**, **accesso con hash**

Accesso Sequenziale



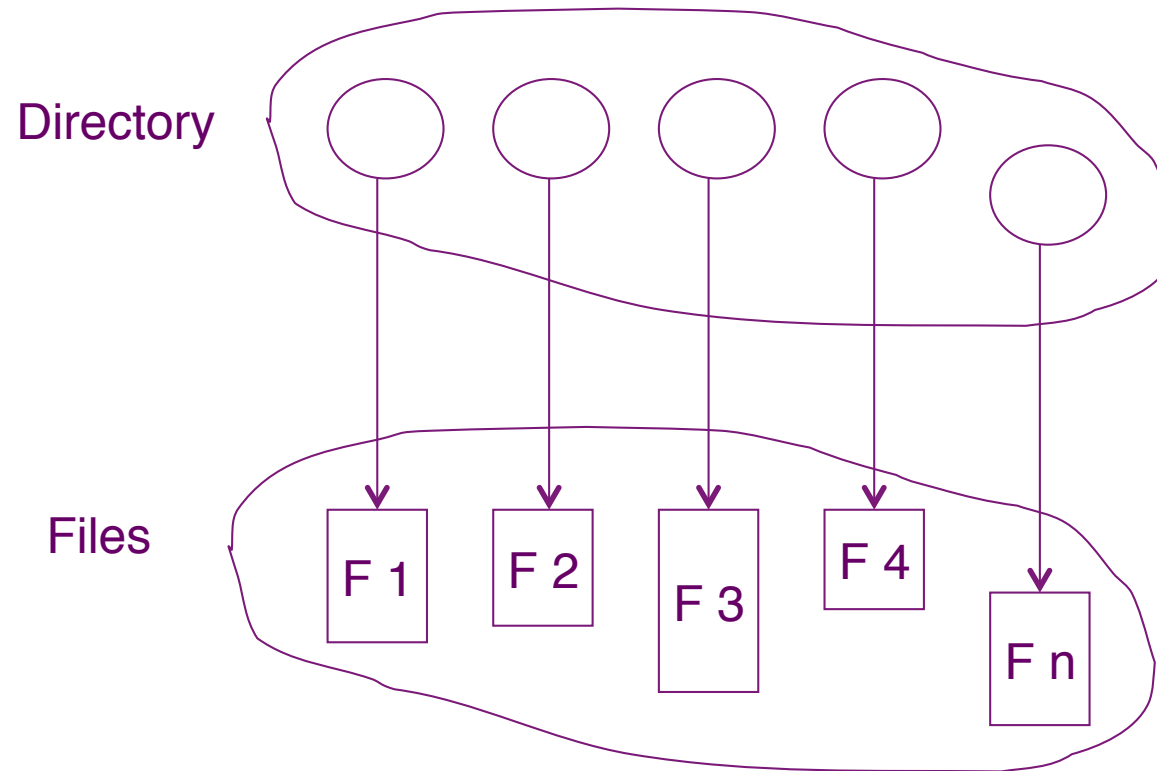
sequential access	implementation for direct access
<i>reset</i>	<i>cp = 0;</i>
<i>read next</i>	<i>read cp;</i> <i>cp = cp+1;</i>
<i>write next</i>	<i>write cp;</i> <i>cp = cp+1;</i>

Accesso con Indice



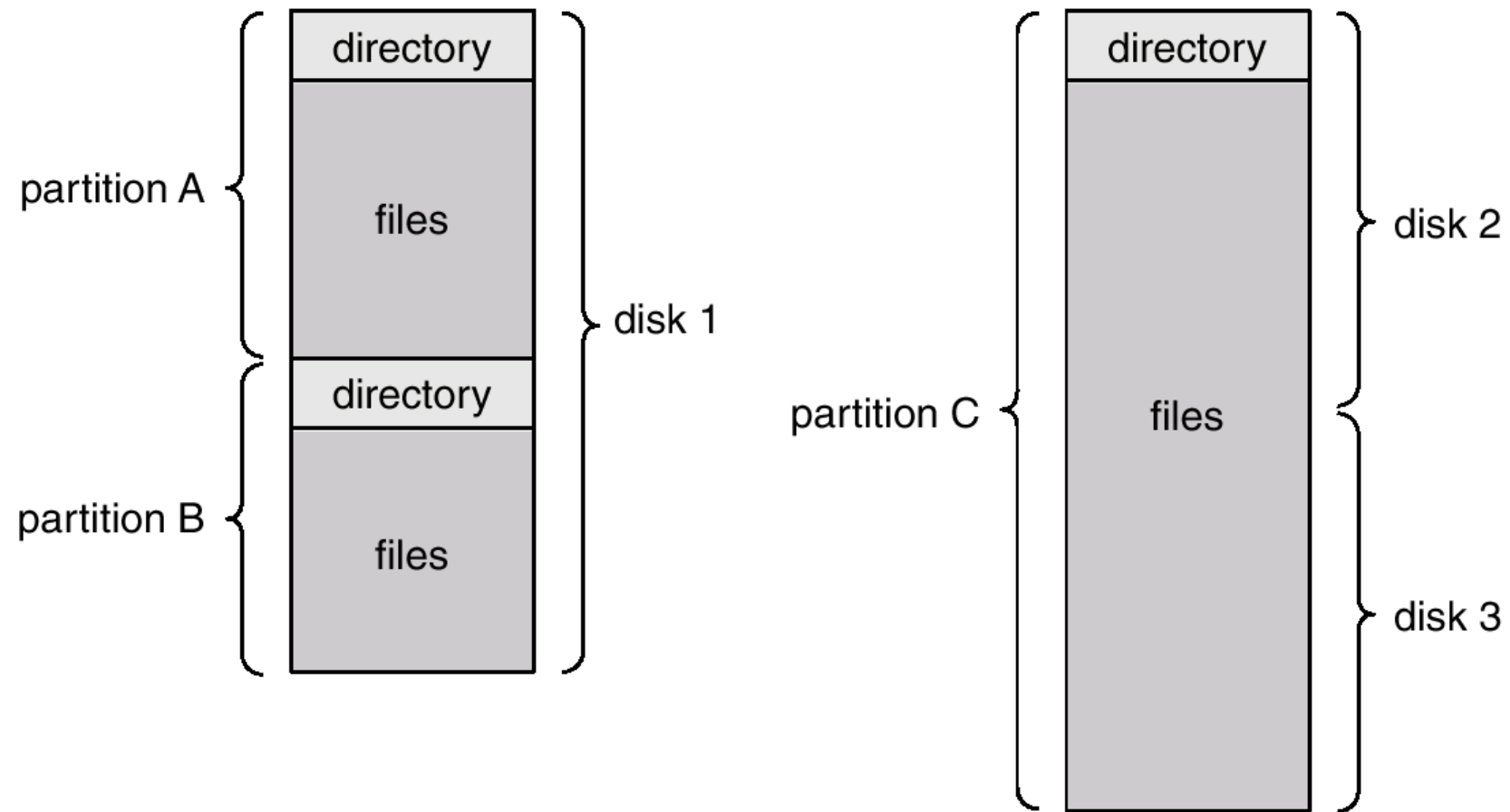
Directory

- È una collezione di file.
- Una directory è un **tipo di dato astratto**. È un tipo di file quindi:
 - Possiede gli stessi attributi e le stesse operazioni dei file.
 - Ha una struttura particolare.
 - Possiede inoltre delle operazioni aggiuntive.





Organizzazione Tipica di un File-System





Elemento di Directory

Ogni elemento di directory contiene informazioni relative ad un file (gli attributi del file)

- Nome
- Tipo
- Locazione
- Dimensione attuale
- Dimensione massima consentita
- Data dell' ultimo accesso
- Data dell' ultimo aggiornamento
- ID del proprietario
- Protezione



Operazioni sulle Directory

- **Search** – ricerca di un file
- **Create** – creazione di un file
- **Delete** – cancellazione di un file
- **List** – elenco di una directory
- **Rename** – ridenominazione di un file
- **Traverse** – attraversamento del file system



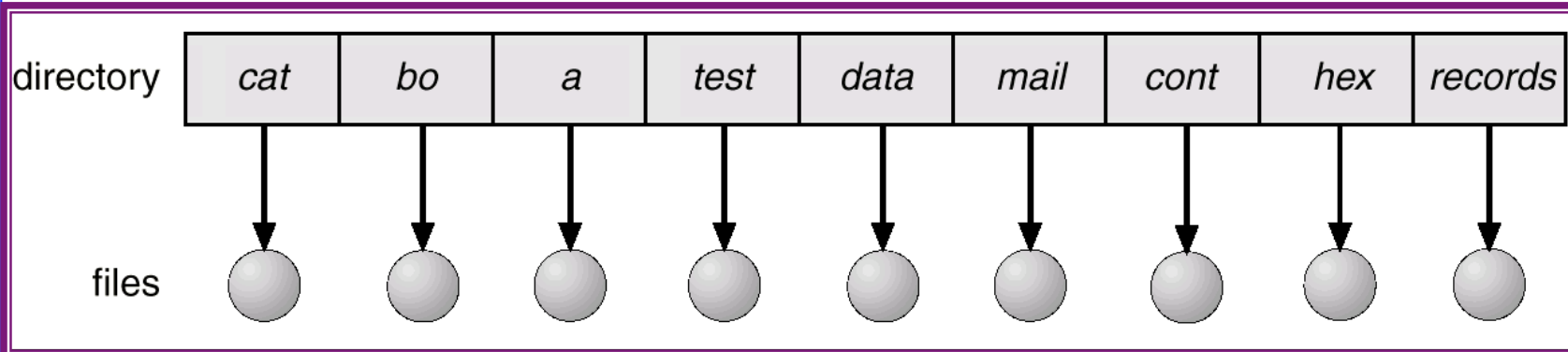
Organizzazione Logica delle Directory

Il file system deve essere organizzato logicamente in modo tale da soddisfare i seguenti requisiti:

- **Efficienza** – deve essere possibile per l'utente ritrovare i propri file il più rapidamente possibile.
- **Assegnazione di nomi unici** – ogni file deve avere un nome unico, ma questo spesso contrasta con le esigenze degli utenti:
 - Due utenti possono usare lo stesso nome per due file diversi.
 - Due utenti possono dare due nomi diversi allo stesso file.
- **Raggruppamento** – deve essere possibile raggruppare logicamente i file sulla base delle loro proprietà (tutti i file di un dato programma, tutti i documenti di un certo tipo, ...).

Directory a Livello Singolo

- Una unica directory per tutti gli utenti.

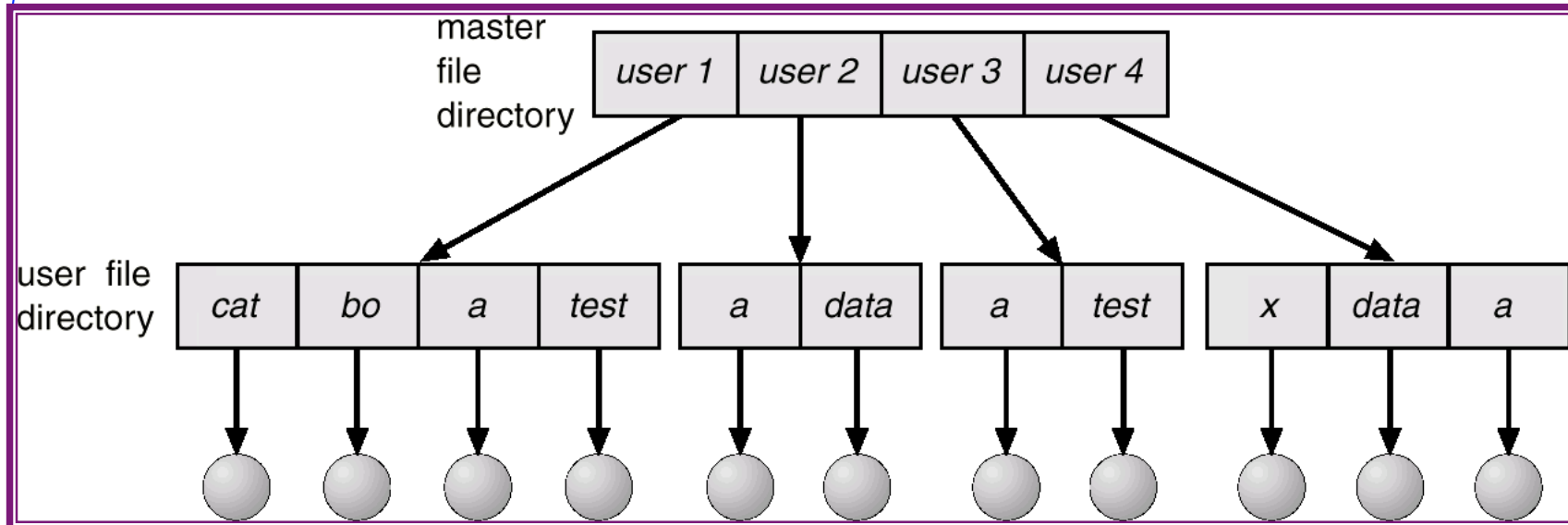


Problema dei nomi unici

Problema del raggruppamento dei file

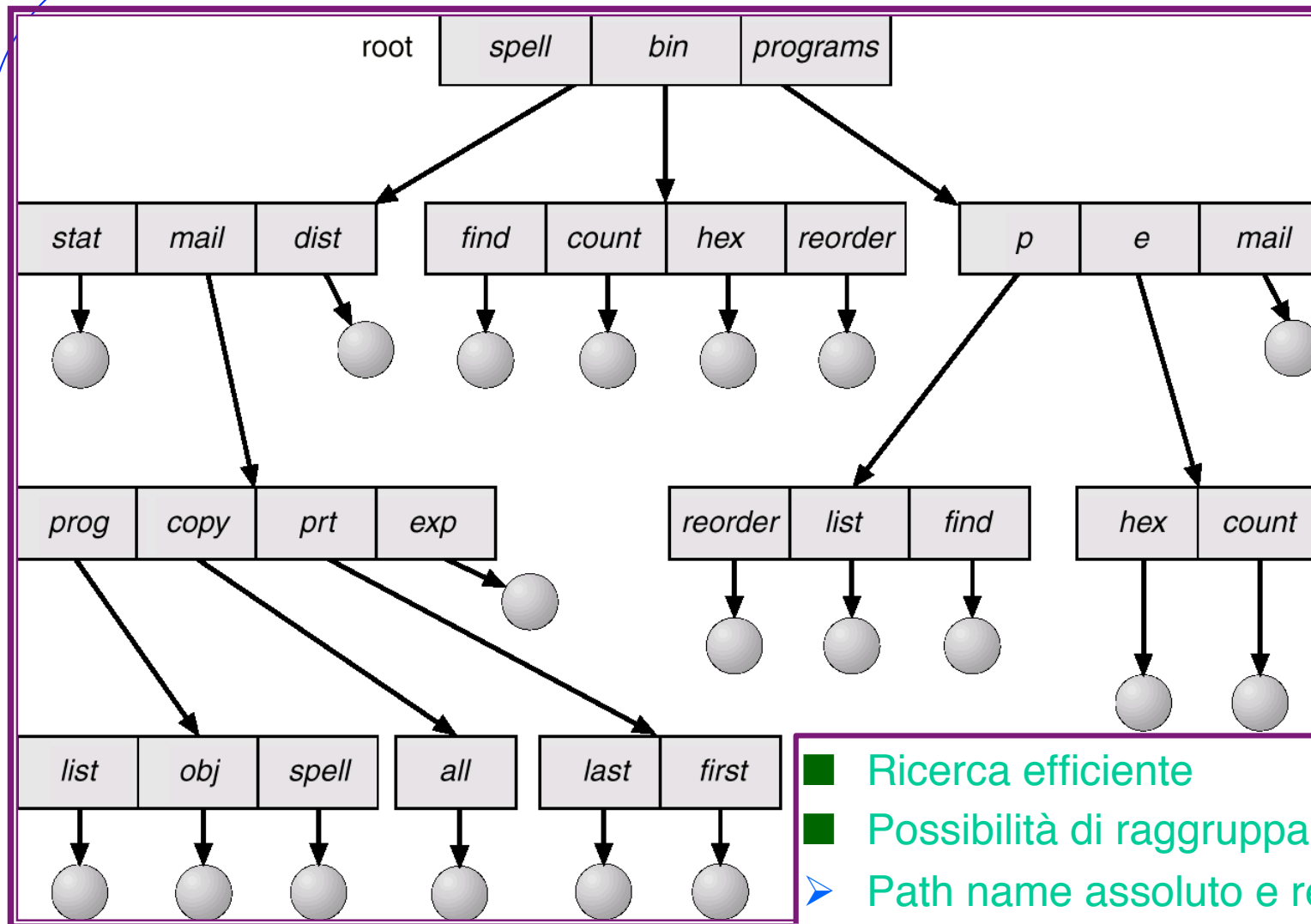
Directory a Due Livelli

- Una directory separata per ogni utente.



- Path name
 - Si possono avere file di utenti diversi con lo stesso nome
 - Ricerca più efficiente
- Ad un utente non è permesso raggruppare i propri file

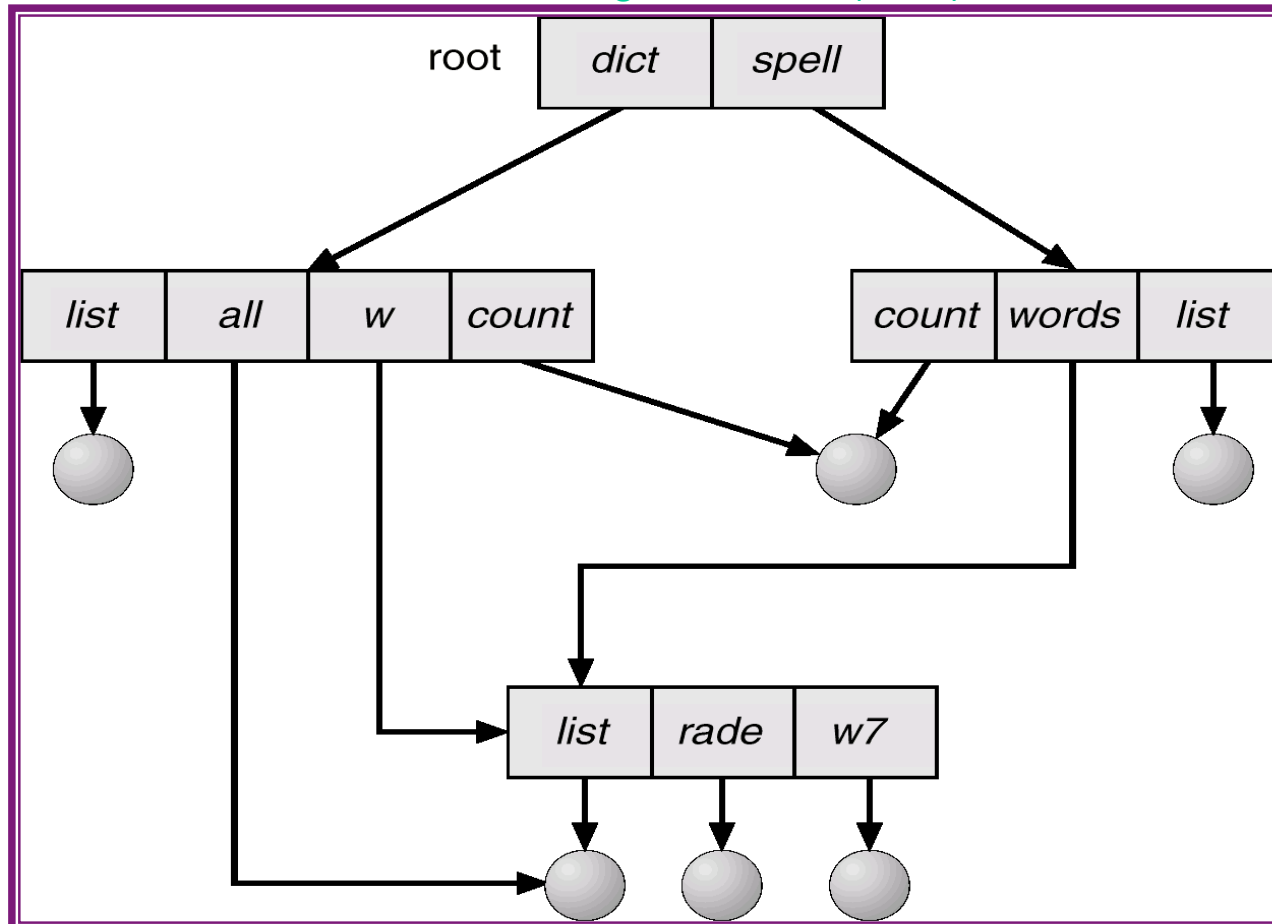
Directory Strutturata ad Albero



- Ricerca efficiente
- Possibilità di raggruppare i file
- Path name assoluto e relativo
- Directory corrente (directory di lavoro)

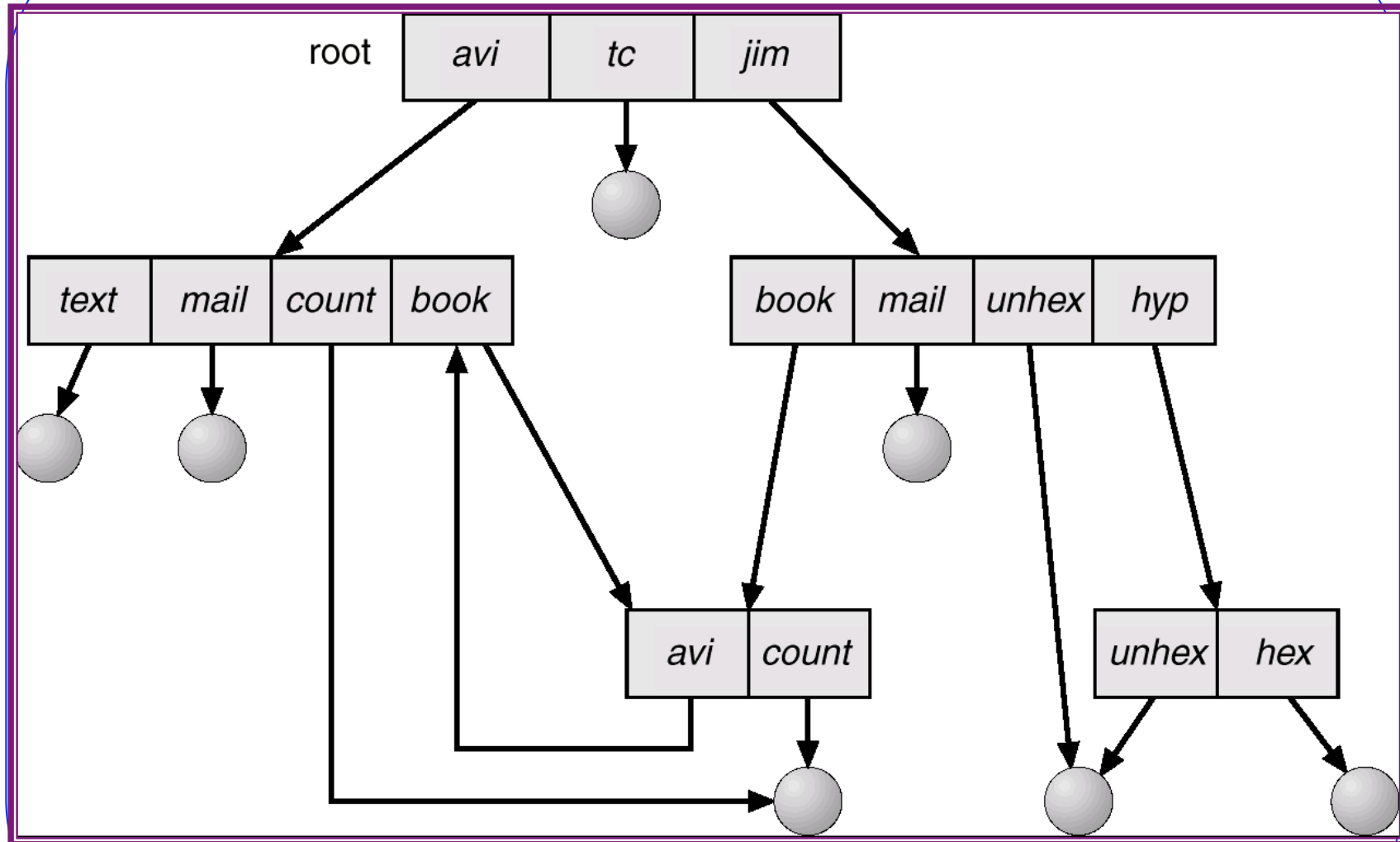
Directory Strutturata a Grafo Aciclico

- È permessa la condivisione di file e directory:
 - Collegamenti simbolici – lo spazio è reso disponibile solo quando è cancellato il file (MS Windows e Unix)
 - Collegamenti effettivi (hard link) – lo spazio è reso disponibile quando
 - ✓ È cancellato il primo hard link
 - ✓ Sono stati cancellati tutti gli hard link (Unix)



Directory Strutturata a Grafo Generale

- È permessa la presenza di cicli



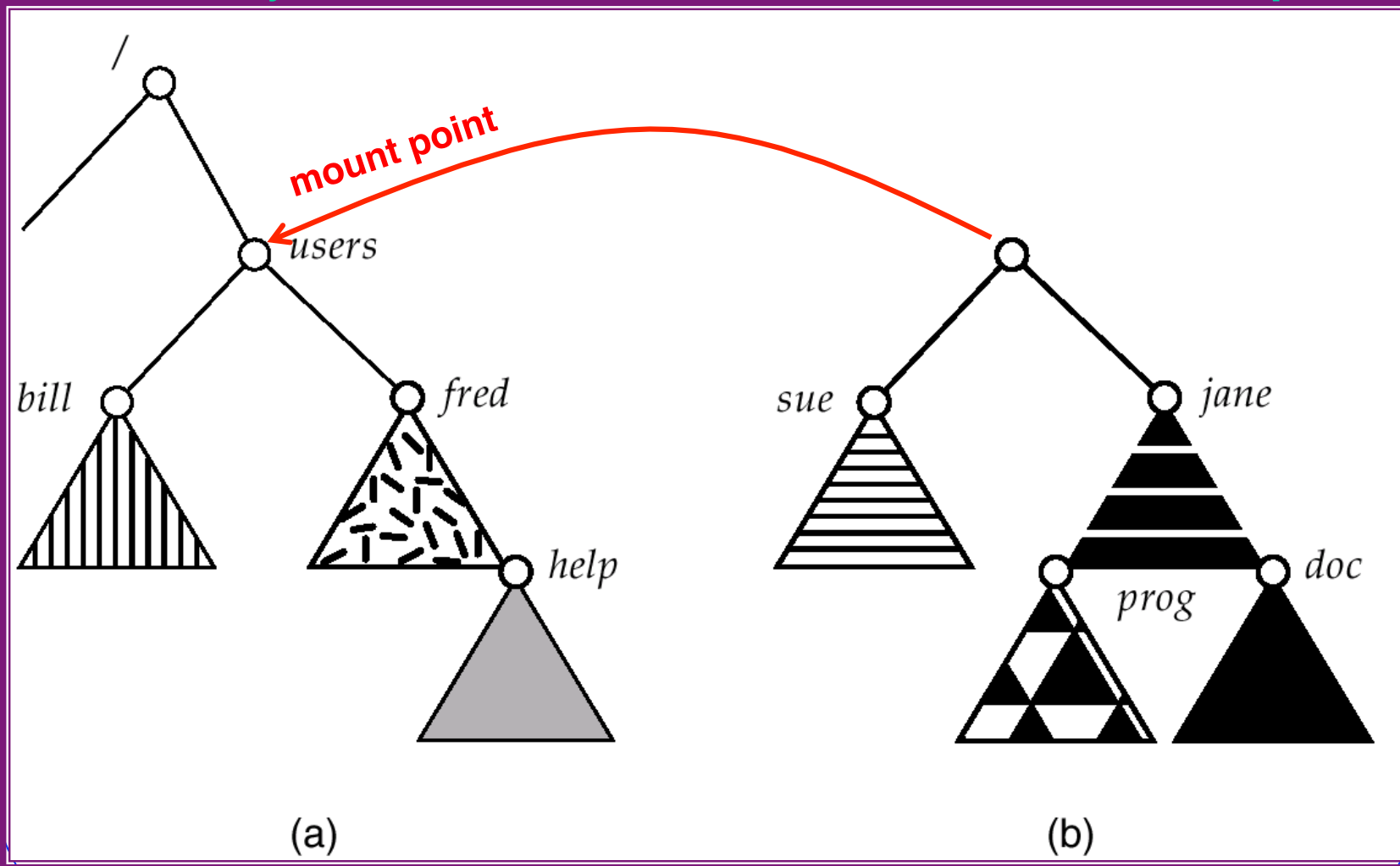


Directory Strutturata a Grafo Generale

- Bisogna evitare che i processi che attraversano il file system vadano in loop infinito:
 - Controllare se si è in presenza di un ciclo nel grafo.
 - Impedire collegamenti a directory
- Sia MS Windows sia Unix permettono cicli realizzati attraverso i collegamenti simbolici.
- Unix non permette hard link a directory.
 - con due eccezioni: `./` `../`

File System Mounting

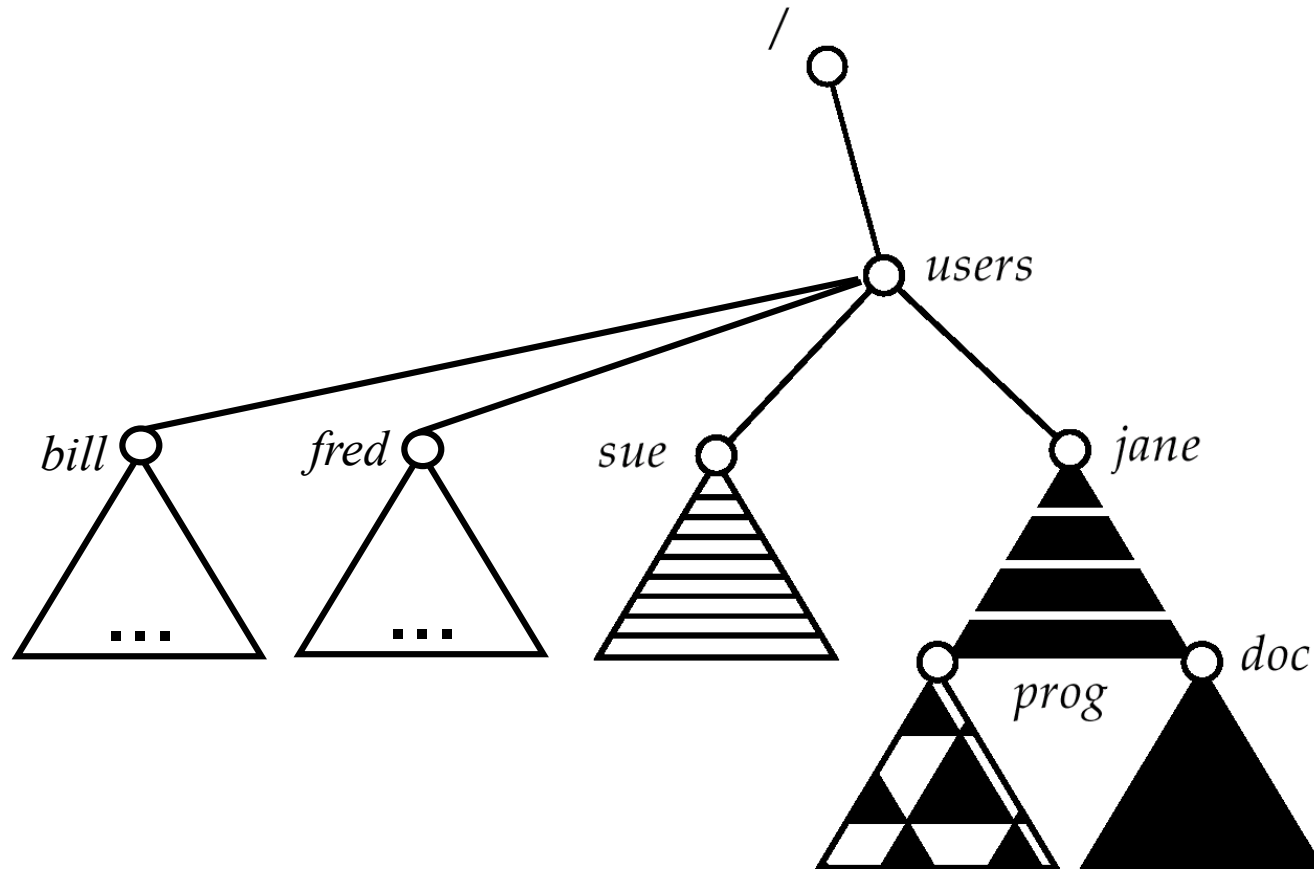
- Un file system deve essere montato prima di potervi accedere.
- Un file system non montato viene montato ad un **mount point**.



File System esistente

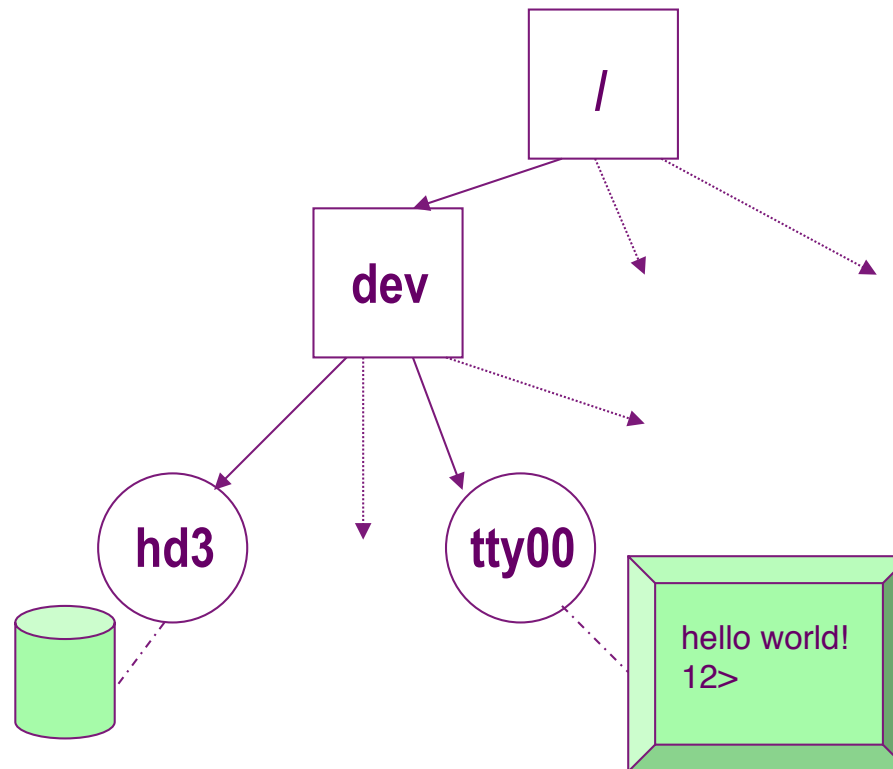
Partizione non montata

File System Mounting



File Speciali

I “file speciali” sono inclusi nella gerarchia delle directory come qualsiasi altro file, ma in realtà corrispondono a device di I/O. Si possono avere file speciali a blocchi (dischi) o a caratteri (tastiere, video a caratteri, ecc.). Questo trucco permette di fare riferimento ad un dispositivo utilizzando un pathname.





Protezioni

- Il proprietario/creatore di un file deve essere in grado di controllare:
 - cosa può essere fatto (modi di accesso: **Read**, **Write**, **eXecute**)
 - da chi

- Gli utenti di ogni file vengono raggruppati in tre categorie:
 - **Proprietario** – RWX
è l'utente che ha creato il file 7 ⇒ 1 1 1
 - **Gruppo** – RW–
insieme di utenti che condividono il file 6 ⇒ 1 1 0
 - **Universo** – – – X
tutti gli altri utenti del sistema 1 ⇒ 0 0 1

- Ogni utente può essere associato ad uno o più gruppi



File-System

✓ Interfaccia del File System

- ✓ Tipo di Dato Astratto File
- ✓ File Ordinari
- ✓ Directory
- ✓ File Speciali
- ✓ Protezioni

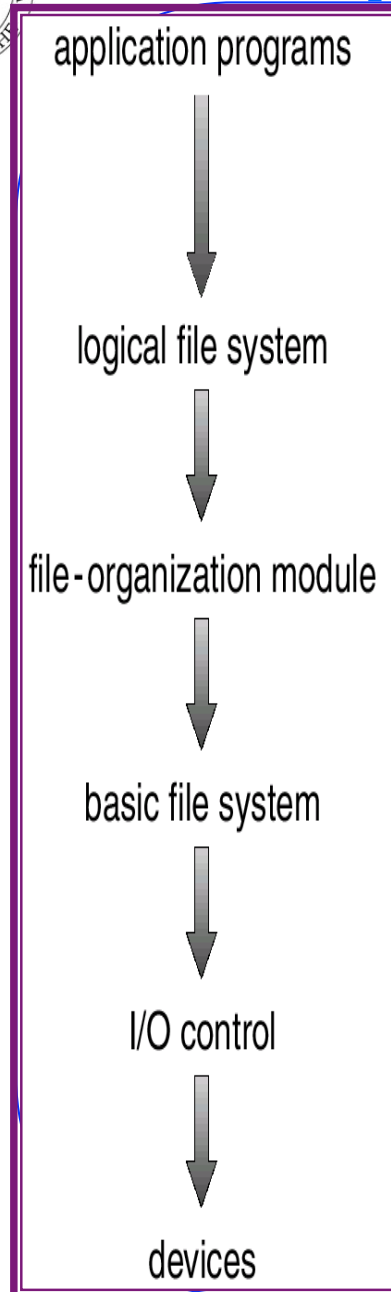
■ Realizzazione del File System

- Struttura dei file system
- Realizzazione delle directory
- Realizzazione dei file (assegnazione dei blocchi liberi)
- Cache (dei blocchi e delle pagine) e Buffering

■ File System di Linux

- Virtual File System
- Ext2 File System
- Ext3 File System
- Proc File System
- Network File System

File System Stratificato



1. Interazione con il file system mediante **system call**. Il file è individuato dal suo **pathname** o dal **FCB handler**. Le librerie di programma trasformano la richiesta di accesso ad un record nella richiesta di accesso ad un dato **blocco logico**.
2. Reperimento dei file all'interno della gerarchia delle directory. Gestione dei metadati (nei descrittori di file o FCB). Es: tipo di file e dispositivo (**carattere** o **blocco**), file location, protezioni, **numero di blocco logico**.
3. Per i dispositivi di tipo blocco; assegnazione dei blocchi liberi del disco; traduzione numeri di blocco logici in **numeri di blocco fisici**.
4. **Caching, buffering, scheduling**. Per i dispositivi di tipo blocco, traduzione dei numeri di blocco fisici in indirizzi del supporto di memorizzazione (**<superficie, cilindro, settore>**). Comunicazione con i driver per leggere e scrivere blocchi fisici o caratteri.
5. **Driver dei dispositivi**. Interazione con i controller (lettura e scrittura nei registri del controller).
6. **Controller**. Interazione con i dispositivi

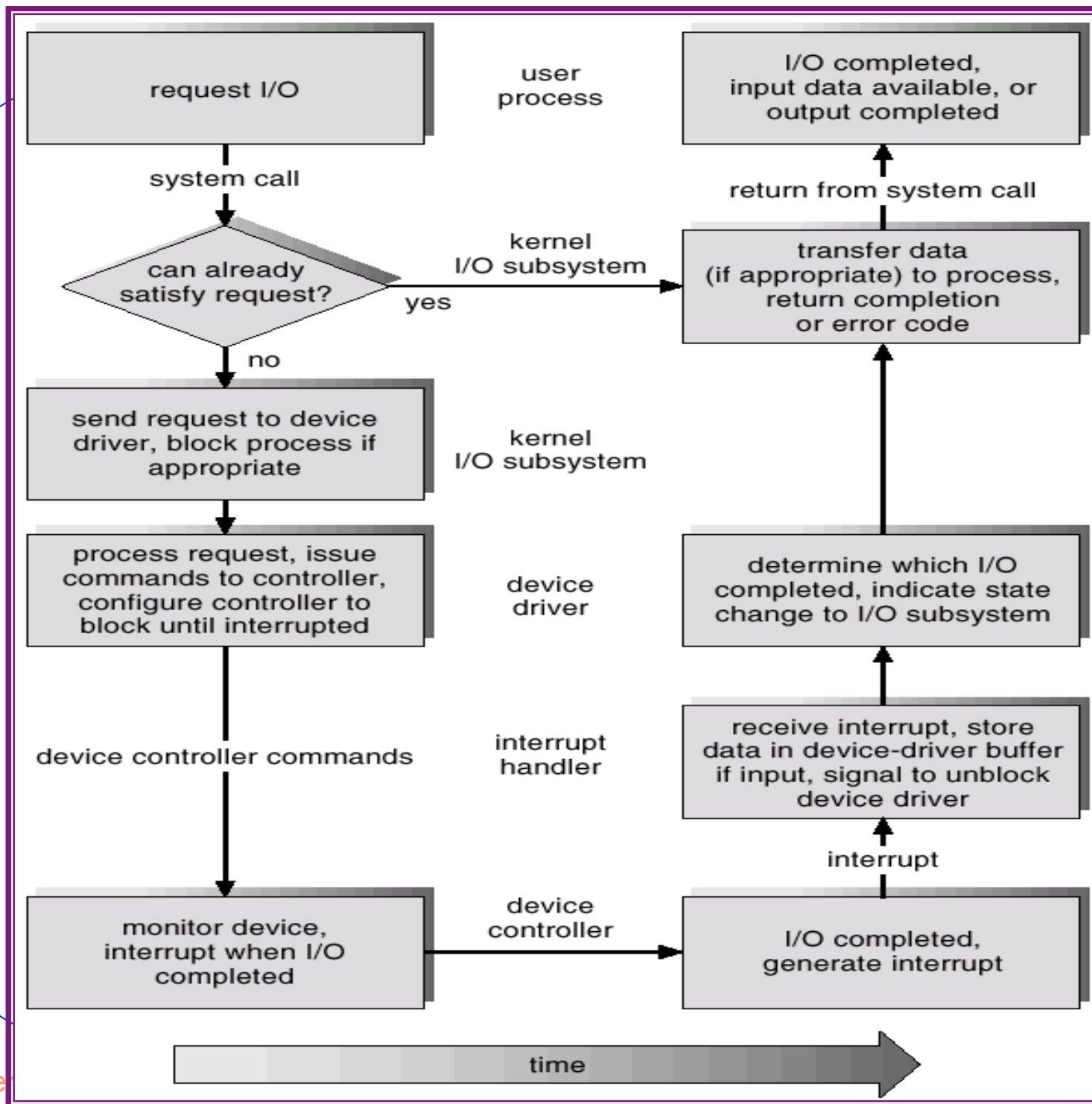


File System Stratificato

Ciascun strato è realizzato grazie ai servizi degli strati sottostanti. Una system call percorrerà i diversi strati a partire dal primo, fino ad arrivare all'ultimo, il driver, che si occuperà di far svolgere al controller del disco su cui si trova il file/directory coinvolto le operazioni necessarie a soddisfare la richiesta.

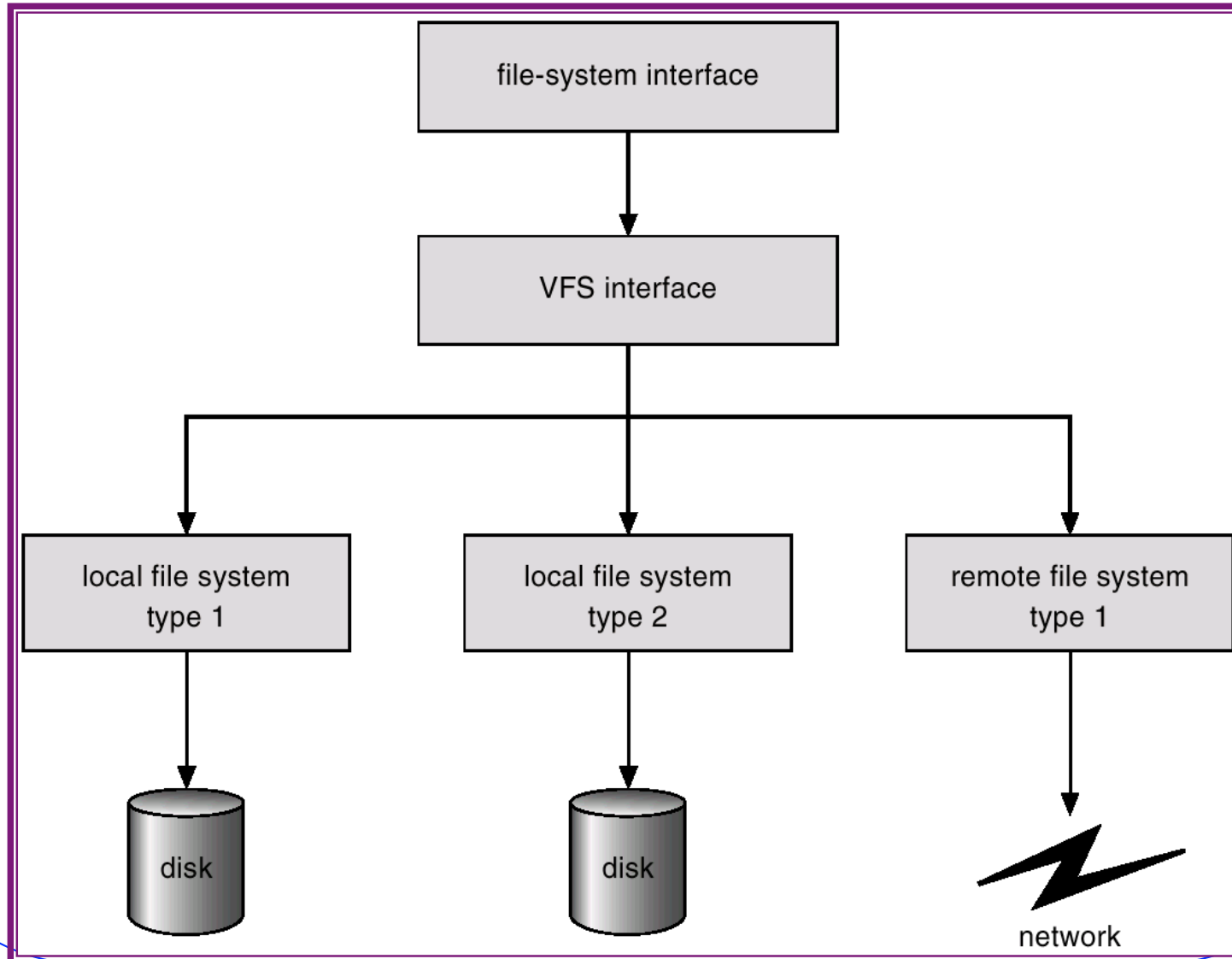
Uno dei motivi per strutturare in questo modo la gestione del file system riguarda la *separazione degli strati che dipendono dall'hardware da quelli che sono indipendenti dall'hardware*.

Ciclo di Vita di una Richiesta di I/O

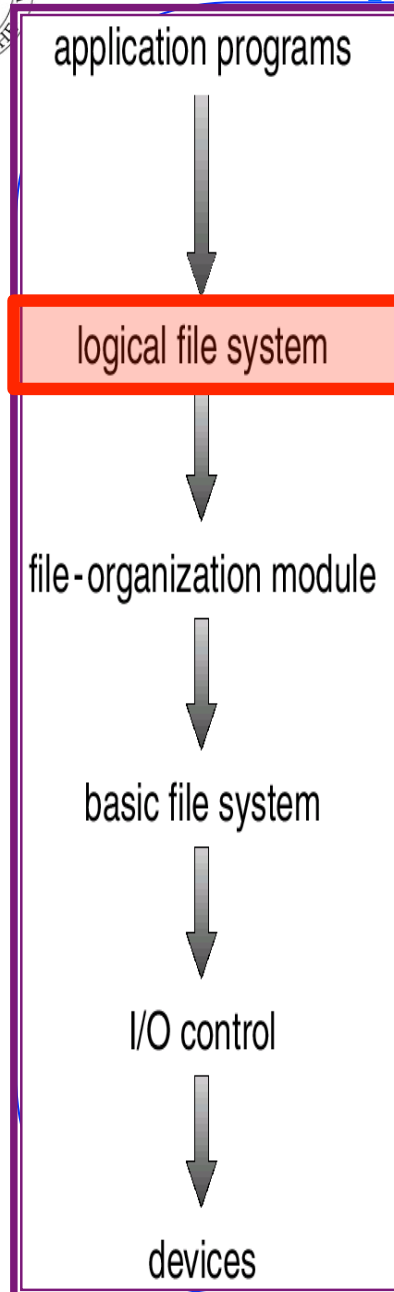


Virtual File System

- VFS fornisce la stessa interfaccia basata su system call (API) da usare per diversi tipi di file system.



File System Stratificato



1. Interazione con il file system mediante **system call**. Il file è individuato dal suo **pathname** o dal **FCB handler**. Le librerie di programma trasformano la richiesta di accesso ad un record nella richiesta di accesso ad un dato **blocco logico**.
2. Reperimento dei file all'interno della gerarchia delle directory. Gestione dei metadati (nei descrittori di file o FCB). Es: tipo di file e dispositivo (**carattere** o **blocco**), file location, protezioni, **numero di blocco logico**.
3. Per i dispositivi di tipo blocco; assegnazione dei blocchi liberi del disco; traduzione numeri di blocco logici in **numeri di blocco fisici**.
4. **Caching, buffering, scheduling**. Per i dispositivi di tipo blocco, traduzione dei numeri di blocco fisici in indirizzi del supporto di memorizzazione (**<superficie, cilindro, settore>**). Comunicazione con i driver per leggere e scrivere blocchi fisici o caratteri.
5. **Driver dei dispositivi**. Interazione con i controller (lettura e scrittura nei registri del controller).
6. **Controller**. Interazione con i dispositivi



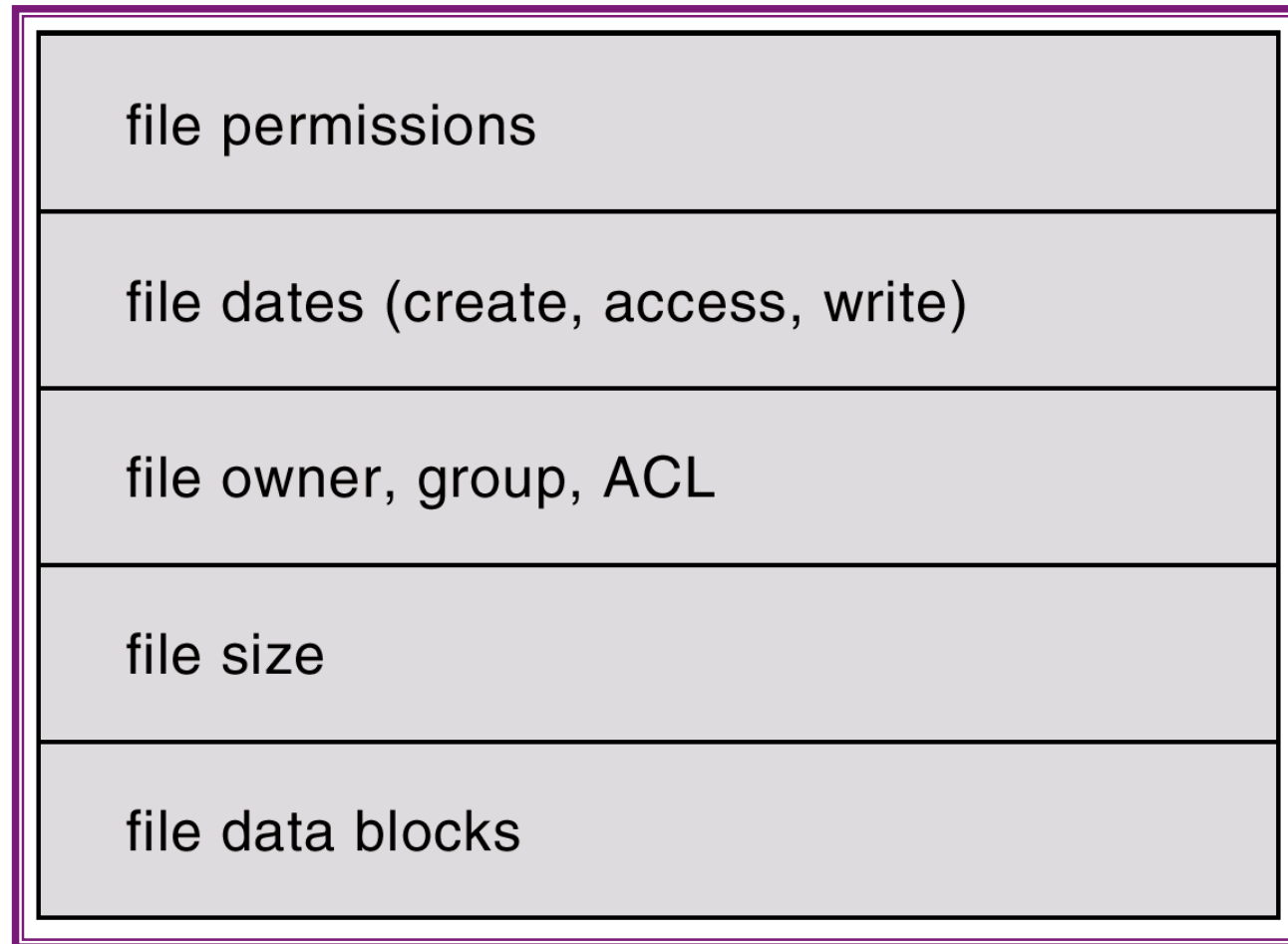
Strutture del File System in Memoria

- Il kernel mantiene le informazioni relative allo stato dei dispositivi di I/O, per esempio la tabella dei file aperti, la tabella delle connessioni di rete, lo stato dei dispositivi a carattere.
- In alcuni SO l'I/O viene realizzato adottando il paradigma ad oggetti e lo scambio dei messaggi, meno efficiente ma di più facile progettazione



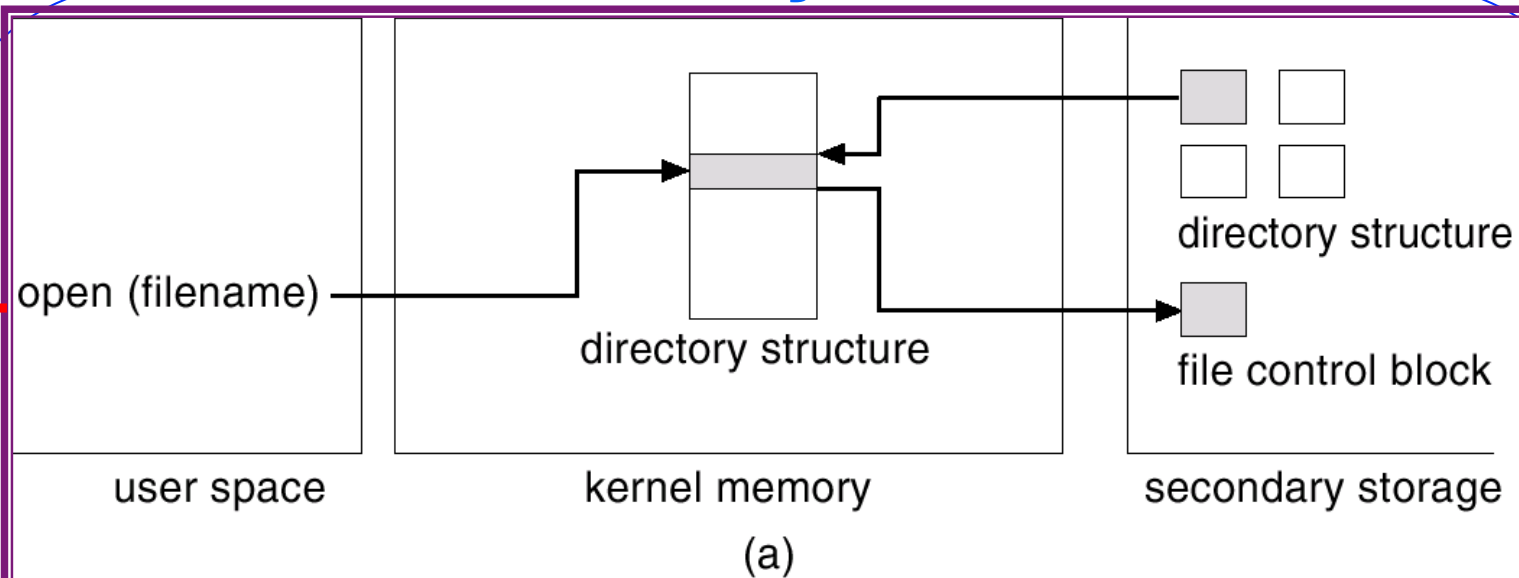
Strutture del File System in Memoria

Descrittore di File - File Control Block

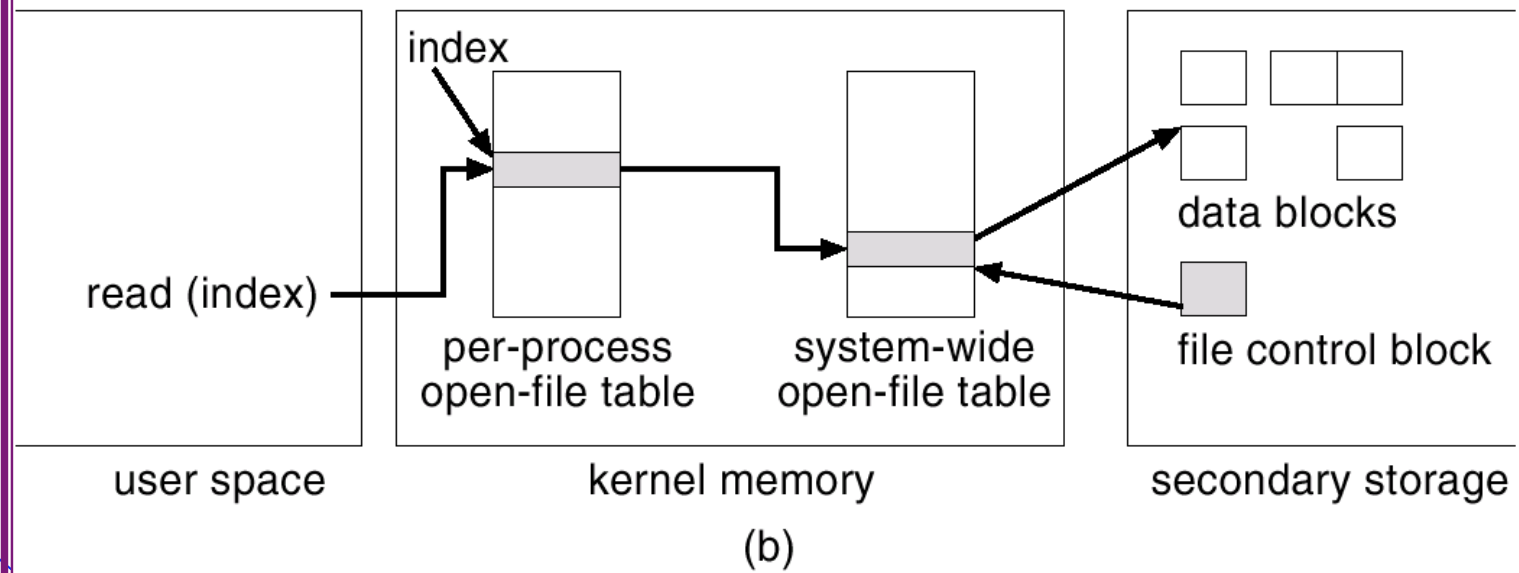


Strutture del File System in Memoria

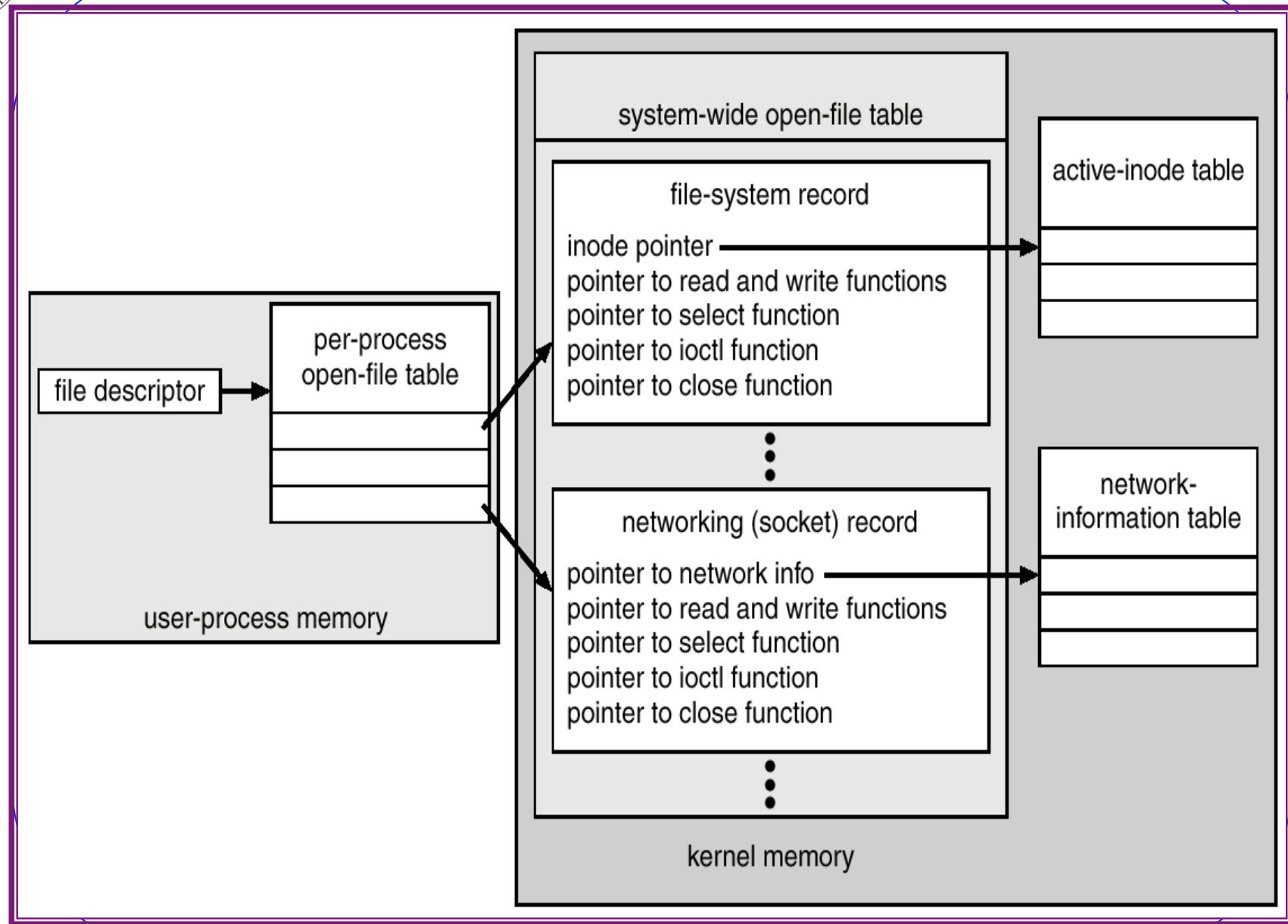
apertura



lettura



Strutture Dati di Unix





Realizzazione delle Directory

■ Lista lineare –

Lista di nomi di file con puntatori ai blocchi dei dati.

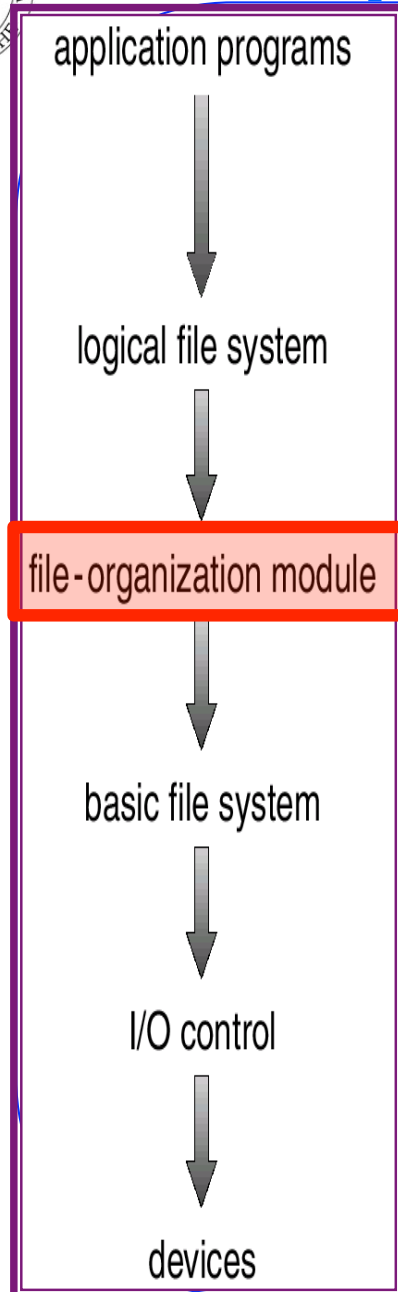
- Ricerca di un file: ricerca lineare.
- Aggiunta di un file: ricerca lineare se non esiste un omonimo ed inserimento in fondo alla lista.
- Cancellazione di un file: ricerca lineare del file, marcatura dell' elemento di directory come libero (oppure si copia l' ultimo elemento in quello liberato oppure si usa una lista concatenata).
- Semplice da programmare.
- Costosa computazionalmente.

■ Hash Table –

Lista lineare con struttura dati hash.

- Ricerca tramite funzione di hash.
- Tempo di ricerca molto breve.
- Bisogna gestire le collisioni (due elementi con lo stesso indice).
- La dimensione della tabella è fissa e decisa a priori.

File System Stratificato



1. Interazione con il file system mediante **system call**. Il file è individuato dal suo **pathname** o dal **FCB handler**. Le librerie di programma trasformano la richiesta di accesso ad un record nella richiesta di accesso ad un dato **blocco logico**.
2. Reperimento dei file all'interno della gerarchia delle directory. Gestione dei metadati (nei descrittori di file o FCB). Es: tipo di file e dispositivo (**carattere** o **blocco**), file location, protezioni, **numero di blocco logico**.
3. Per i dispositivi di tipo blocco; assegnazione dei blocchi liberi del disco; traduzione numeri di blocco logici in **numeri di blocco fisici**.
4. **Caching, buffering, scheduling**. Per i dispositivi di tipo blocco, traduzione dei numeri di blocco fisici in indirizzi del supporto di memorizzazione (**<superficie, cilindro, settore>**). Comunicazione con i driver per leggere e scrivere blocchi fisici o caratteri.
5. **Driver dei dispositivi**. Interazione con i controller (lettura e scrittura nei registri del controller).
6. **Controller**. Interazione con i dispositivi



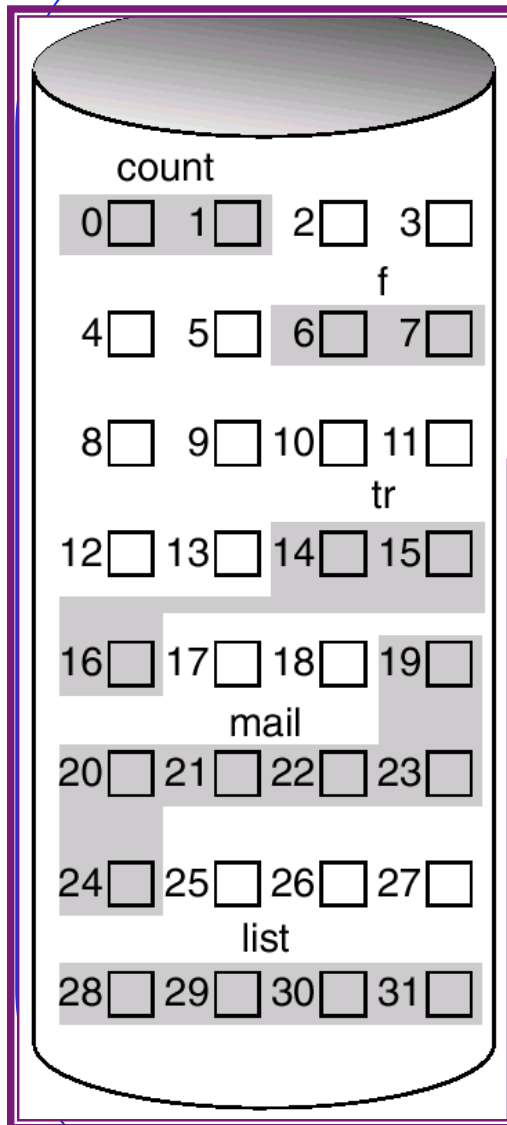
Metodi di Allocazione

Il problema principale da risolvere è come assegnare lo spazio ai file

- Assegnazione Contigua
- Assegnazione Concatenata
- Assegnazione Indicizzata

Assegnazione Contigua

Ogni file occupa un insieme contiguo di blocchi nel disco.



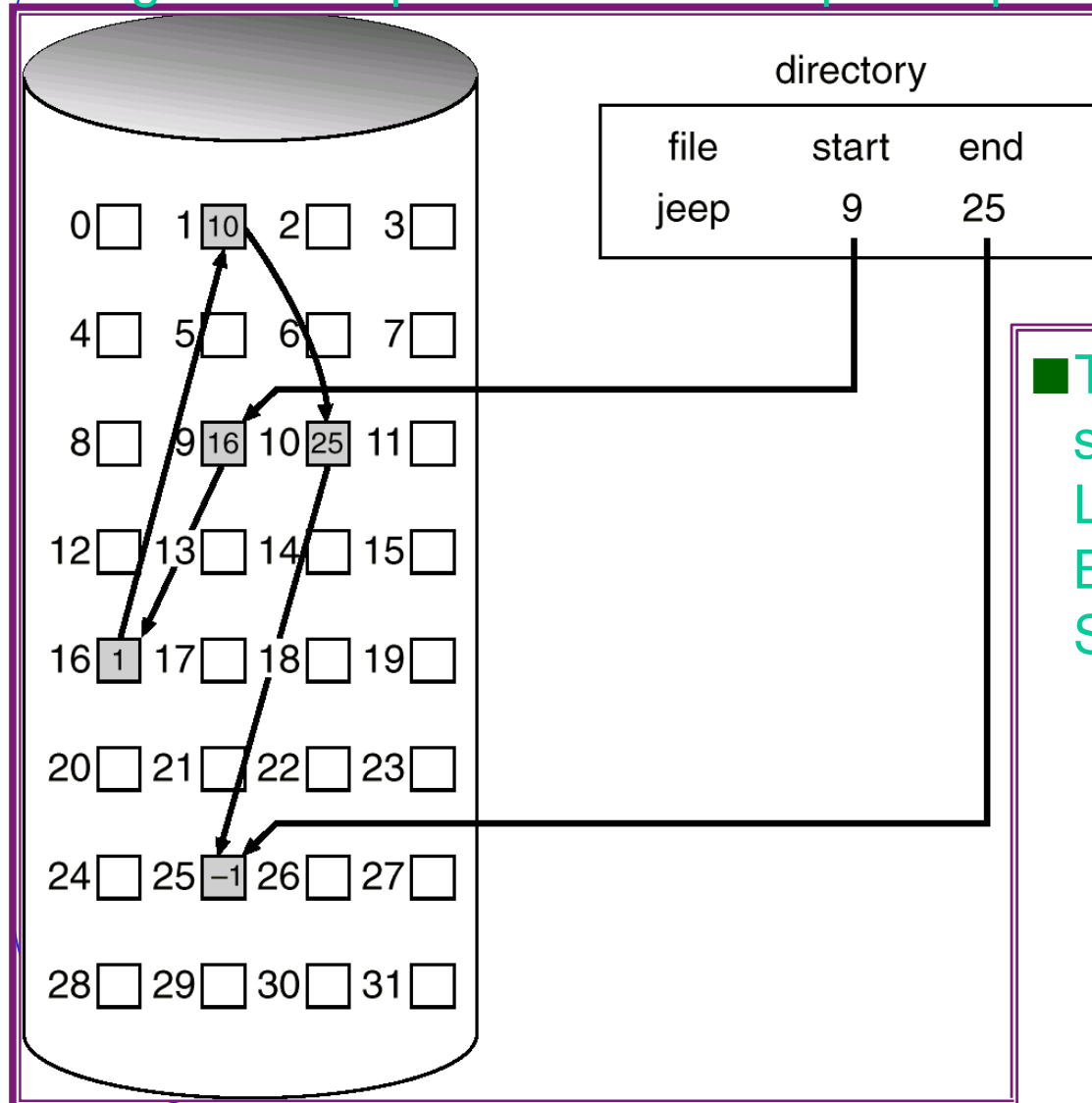
directory		
file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

- ✓ È semplice, serve solo la locazione iniziale e la lunghezza (numero dei blocchi) .
- ✓ Consente l' accesso diretto.
- Problema dell' assegnazione dinamica della memoria:
 - first-fit, best-fit, worst-fit,
 - frammentazione esterna.
- Problema della quantità di spazio necessario:
 - La dimensione massima è fissa.
 - Frammentazione interna
 - Assegnazione contigua con **Estensione**

Assegnazione Concatenata

Ogni file è una lista concatenata di blocchi.

Ogni blocco può trovarsi in un punto qualsiasi del disco.



■ Traduzione degli indirizzi
sia

LA = Indirizzo Logico

B = Numero di Blocco

S = Scostamento:

■ $B = LA / 511$

■ $S = (LA \% 511) + 1$

Assegnazione Concatenata

- ✓ È semplice.
- ✓ Non esiste frammentazione esterna.
- ✓ Non è necessario dichiarare la dimensione massima.

➤ Permette solo l'accesso sequenziale.

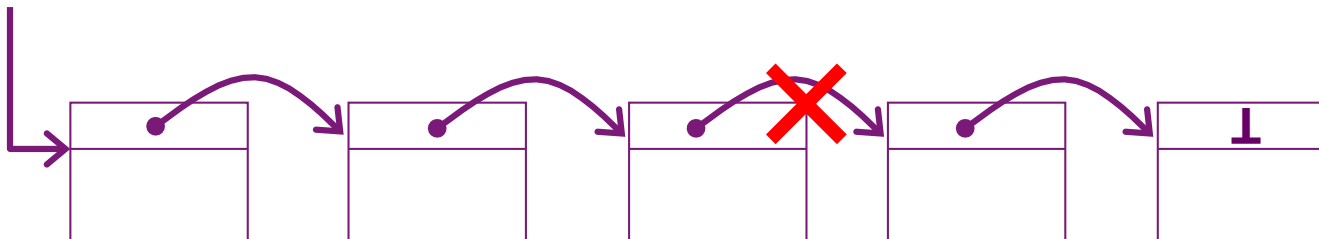
➤ I puntatori richiedono spazio.

Blocco =
(512 byte)

Pointer: 4 byte

Dati: 508 byte

➤ Affidabilità (se si perde un puntatore ...)

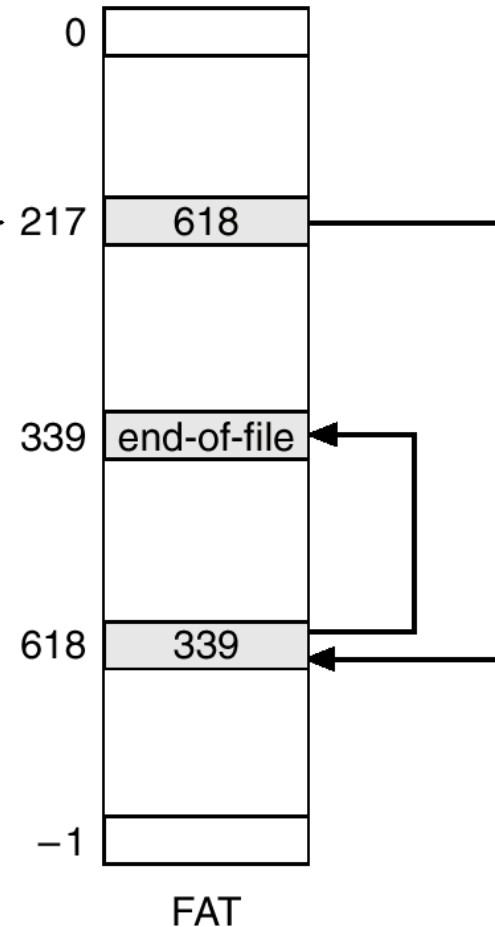
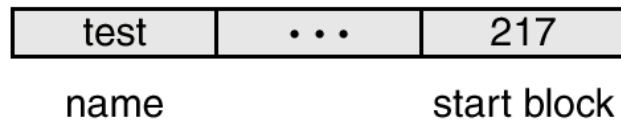


File-Allocation Table (FAT)

Variante dell'assegnazione concatenata.

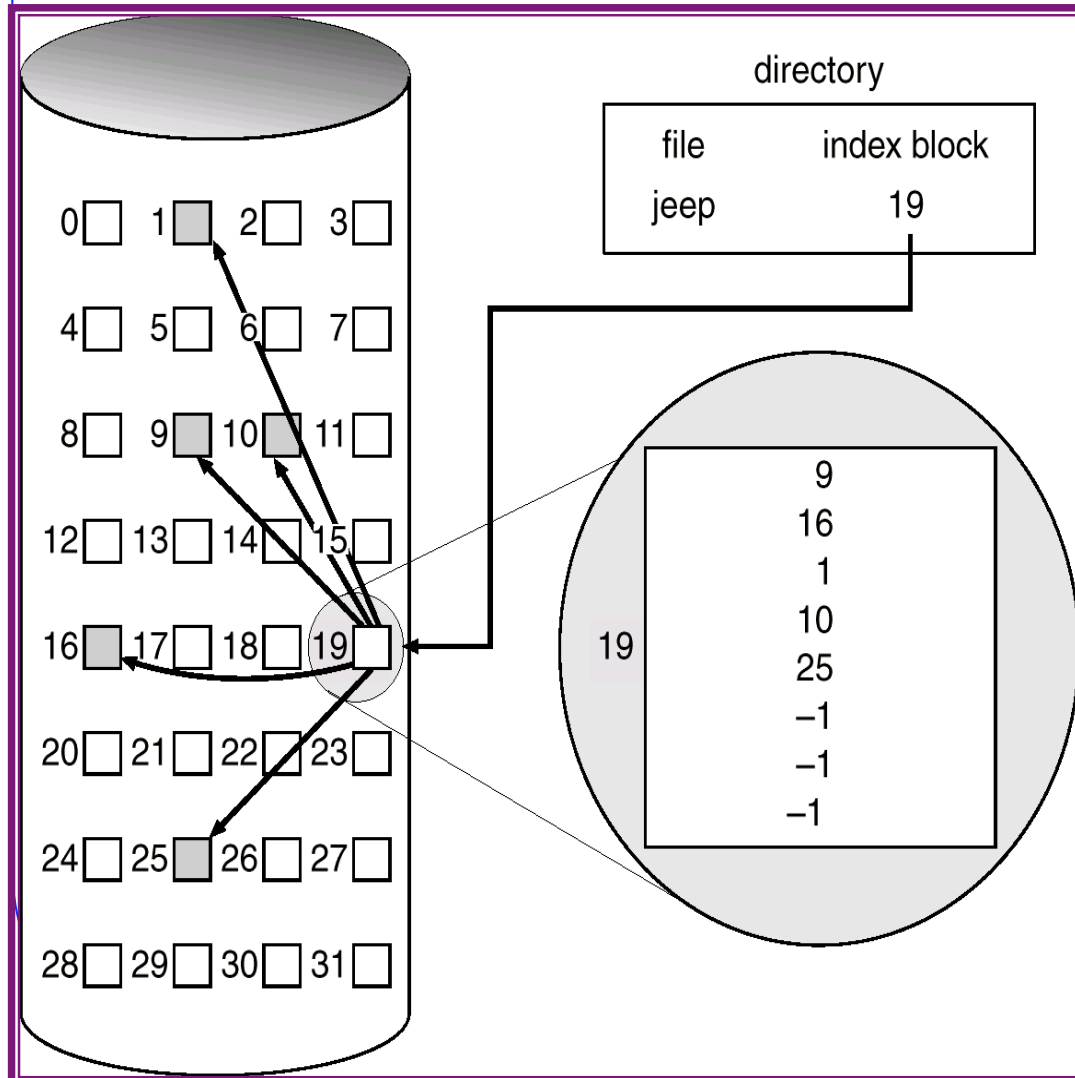
È utilizzata da MS-DOS e OS/2.

directory entry



Assegnazione Indicizzata

Tutti i puntatori vengono mantenuti in una **tabella indice**.



- ✓ Accesso diretto
- ✓ Accesso dinamico senza frammentazione esterna.
- ✓ File di lunghezza illimitata.
- Necessità di riservare dello spazio per la tabella indice.
- Sovraccarico per la gestione della tabella indice.



Assegnazione Indicizzata

Schema Concatenato

- La tabella degli indici è realizzata con una **lista concatenata di blocchi**
- Traduzione degli indirizzi
sia
LA = Indirizzo Logico
 Q_1 = blocco della tabella indice
 Q_2 = scostamento nel blocco della tabella indice
 R_2 = scostamento nel blocco del file
 - $Q_1 = LA / (512 \times 511)$
 - $R_1 = LA \% (512 \times 511)$
 - $Q_2 = R_1 / 512$
 - $R_2 = R_1 \% 512$

Assegnazione Indicizzata

Indice a più Livelli

- La tabella degli indici è a sua volta indicizzata.
- Traduzione degli indirizzi (esempio a due livelli)
sia

LA = Indirizzo Logico

Q_1 = scostamento nella tabella indice di primo livello (outer index)

Q_2 = scostamento nel blocco della tabella indice di 2° livello

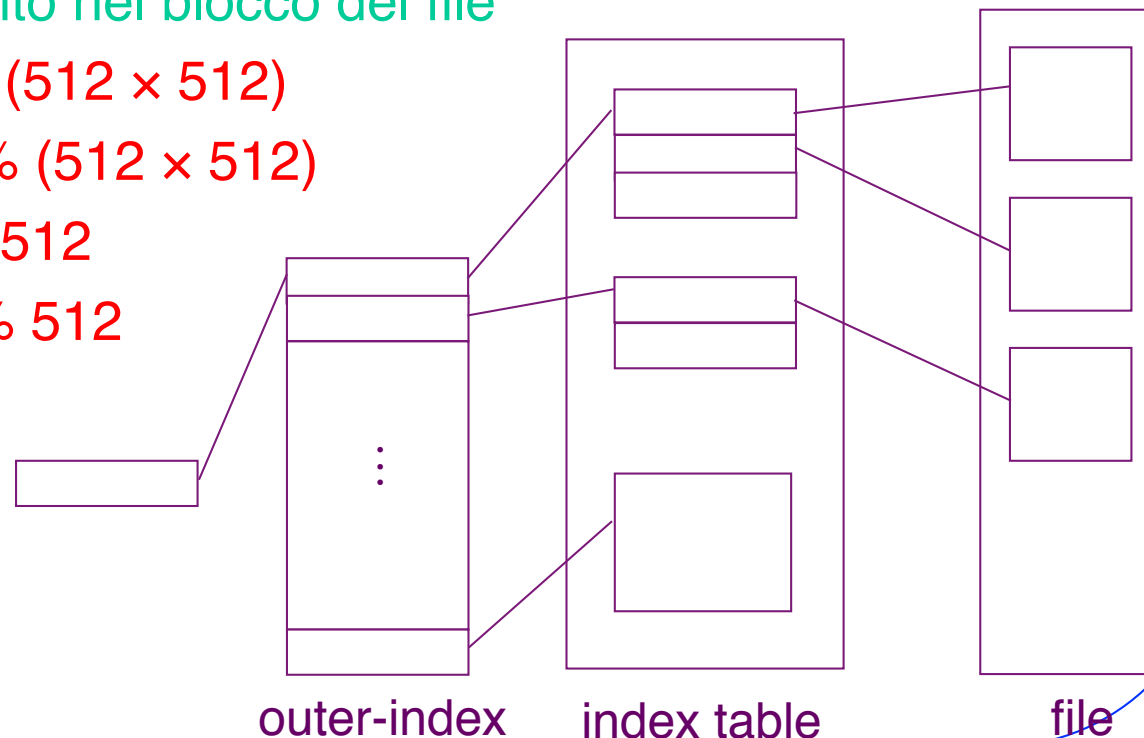
R_2 = scostamento nel blocco del file

- $Q_1 = LA / (512 \times 512)$

- $R_1 = LA \% (512 \times 512)$

- $Q_2 = R_1 / 512$

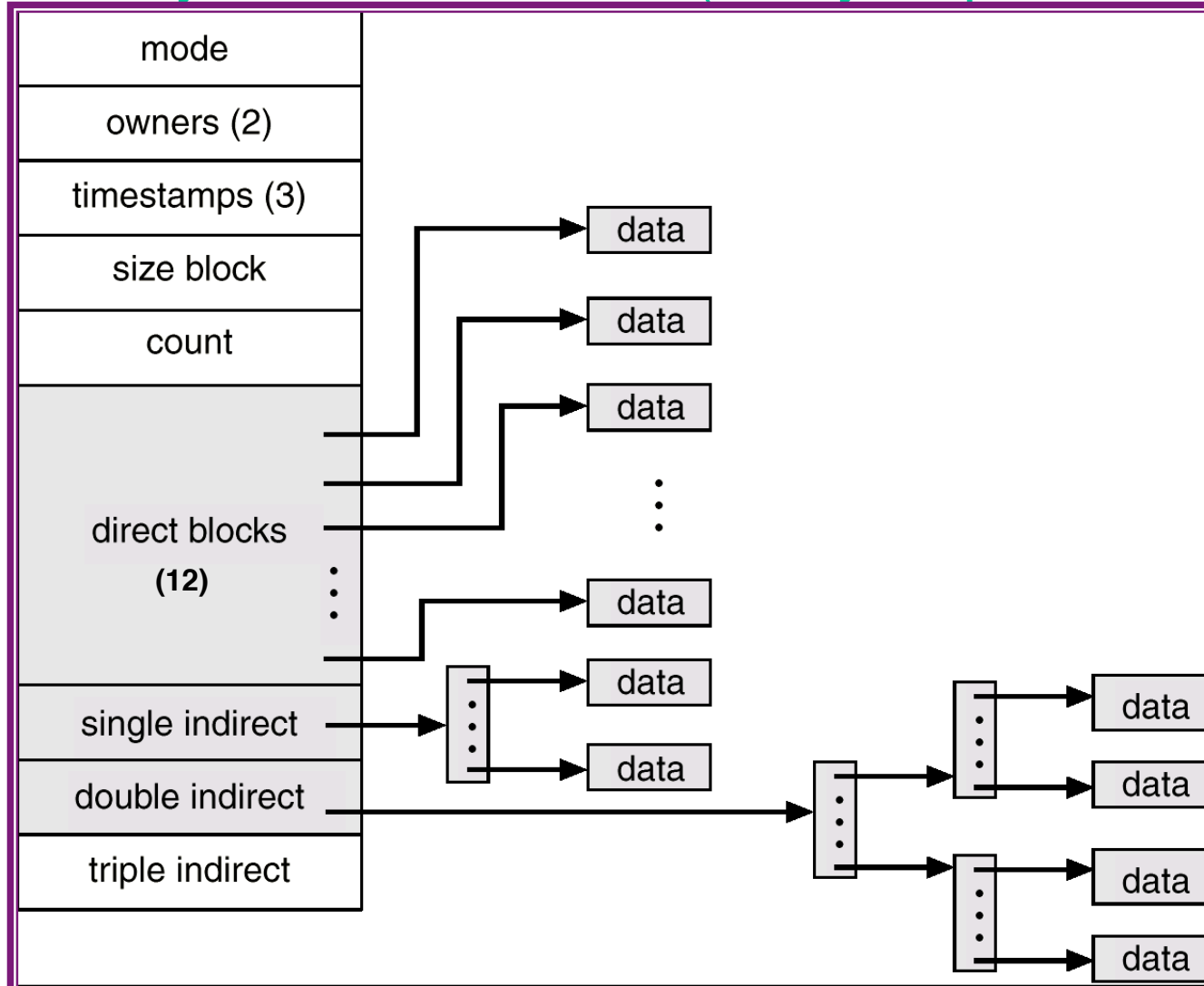
- $R_2 = R_1 \% 512$



Assegnazione Indicizzata

Schema Combinato

UFS – File System di Unix BSD (8K bytes per blocco)



Gestione dello Spazio Libero

Vettore di Bit



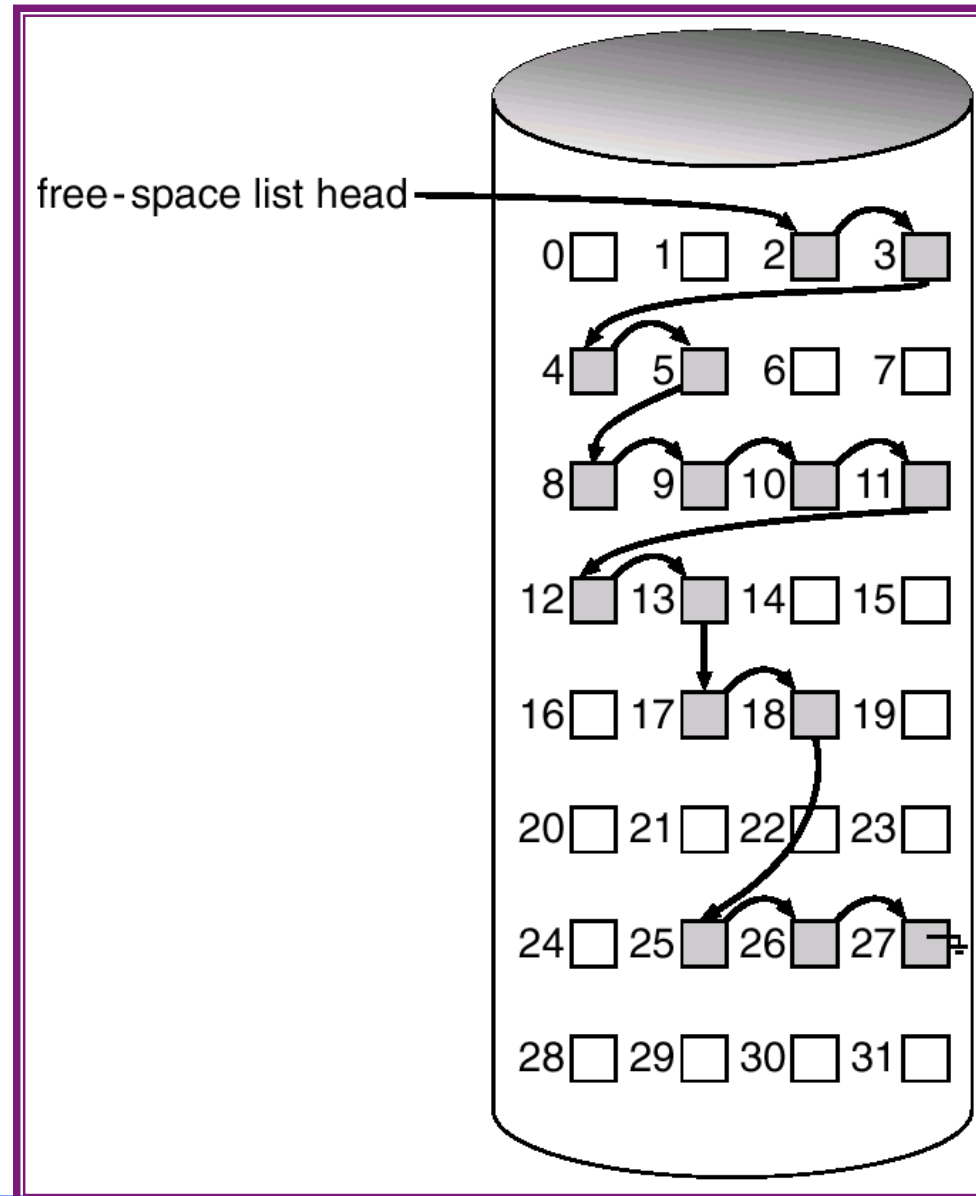
$$\text{bit}[i] = \begin{cases} 0 \Rightarrow \text{block}[i] \text{ libero} \\ 1 \Rightarrow \text{block}[i] \text{ occupato} \end{cases}$$

numero blocco = (numero di bit per parola) * (numero di parole di valore 0) + (offset del primo bit a 1)

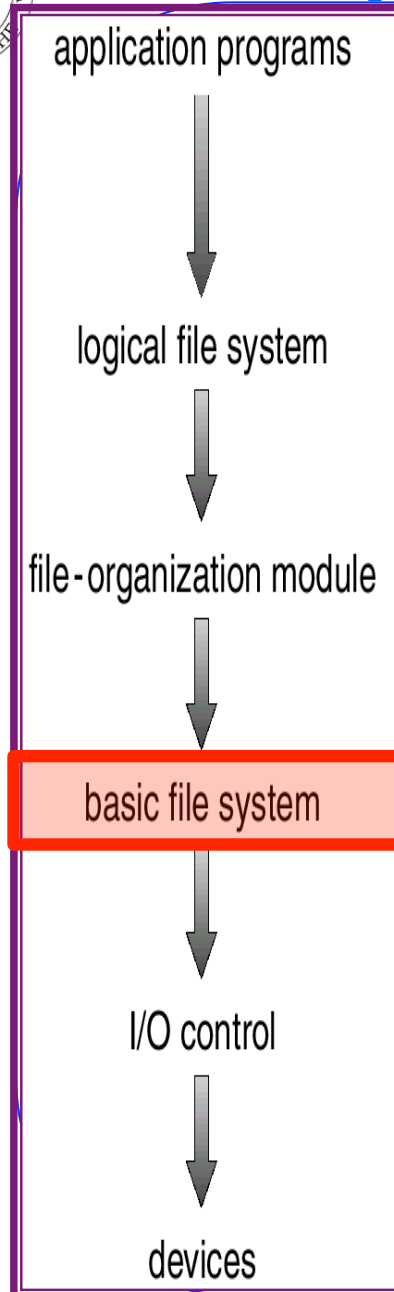
- ✓ Facile da realizzare
- ✓ Efficiente solo se il vettore è tenuto in memoria centrale
- Utilizzabile solo se i dischi sono piccoli

Gestione dello Spazio Libero

Lista Concatenata



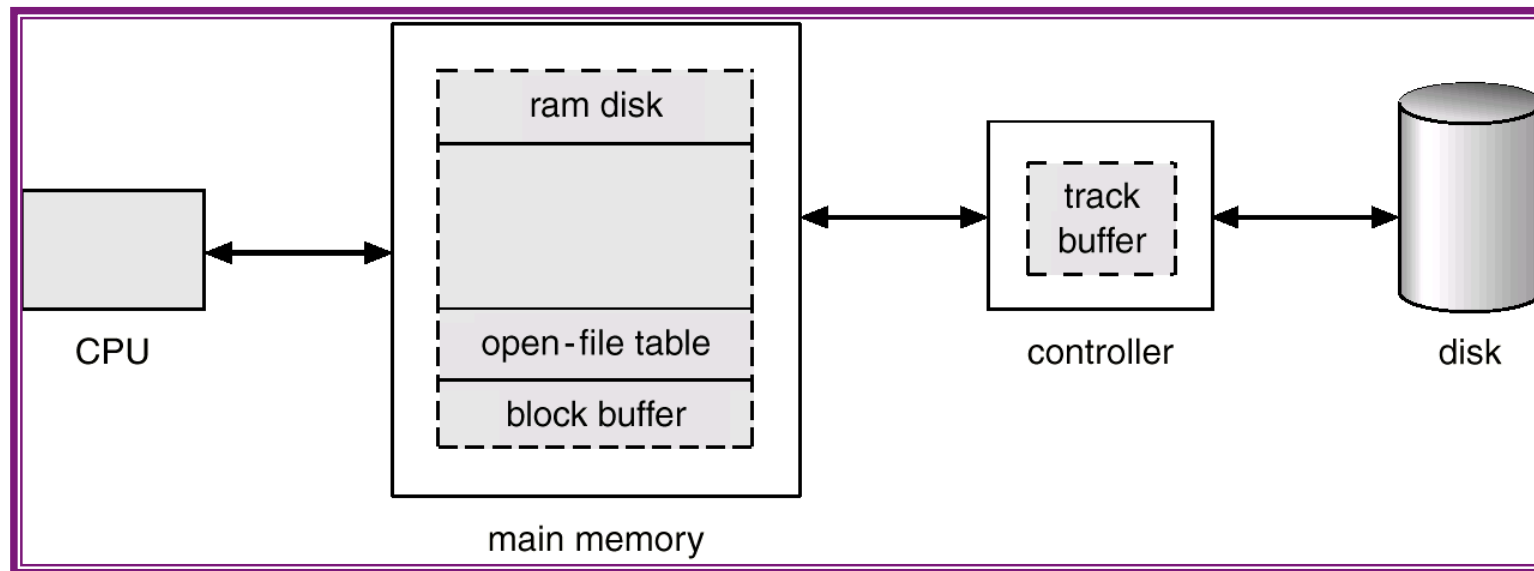
File System Stratificato



1. Interazione con il file system mediante **system call**. Il file è individuato dal suo **pathname** o dal **FCB handler**. Le librerie di programma trasformano la richiesta di accesso ad un record nella richiesta di accesso ad un dato **blocco logico**.
2. Reperimento dei file all'interno della gerarchia delle directory. Gestione dei metadati (nei descrittori di file o FCB). Es: tipo di file e dispositivo (**carattere** o **blocco**), file location, protezioni, **numero di blocco logico**.
3. Per i dispositivi di tipo blocco; assegnazione dei blocchi liberi del disco; traduzione numeri di blocco logici in **numeri di blocco fisici**.
4. **Caching, buffering, scheduling**. Per i dispositivi di tipo blocco, traduzione dei numeri di blocco fisici in indirizzi del supporto di memorizzazione (**<superficie, cilindro, settore>**). Comunicazione con i driver per leggere e scrivere blocchi fisici o caratteri.
5. **Driver dei dispositivi**. Interazione con i controller (lettura e scrittura nei registri del controller).
6. **Controller**. Interazione con i dispositivi

Cache del Disco

- Per migliorare le prestazioni del file system si può utilizzare una parte della memoria centrale come cache del disco.
 - Nella cache del disco vengono tenuti i blocchi che si prevede di utilizzare entro breve tempo
 - Per migliorare le prestazioni nei PC, a volte una parte della memoria viene utilizzata come **disco virtuale (RAM disk)**.
- Algoritmo di sostituzione dei blocchi: **LRU, LFU**



Diverse locazioni di cache per i dischi



Cache delle Pagine

- Utilizza tecniche di memoria virtuale per la gestione dei dati dei file come pagine anziché come blocchi fisici del disco.
 - L'uso degli indirizzi virtuali è più efficiente dell'uso dei blocchi fisici del disco
 - Questo permette di utilizzare la cache sia per le pagine dei processi sia per le pagine dei dati del disco (memoria virtuale unificata)
- Algoritmo di sostituzione delle pagine:
- LRU, LFU
 - **Paginazione con priorità**
l'algoritmo di scansione delle pagine dà la priorità alle pagine dei processi rispetto a quelle dei dati del disco.

Caching

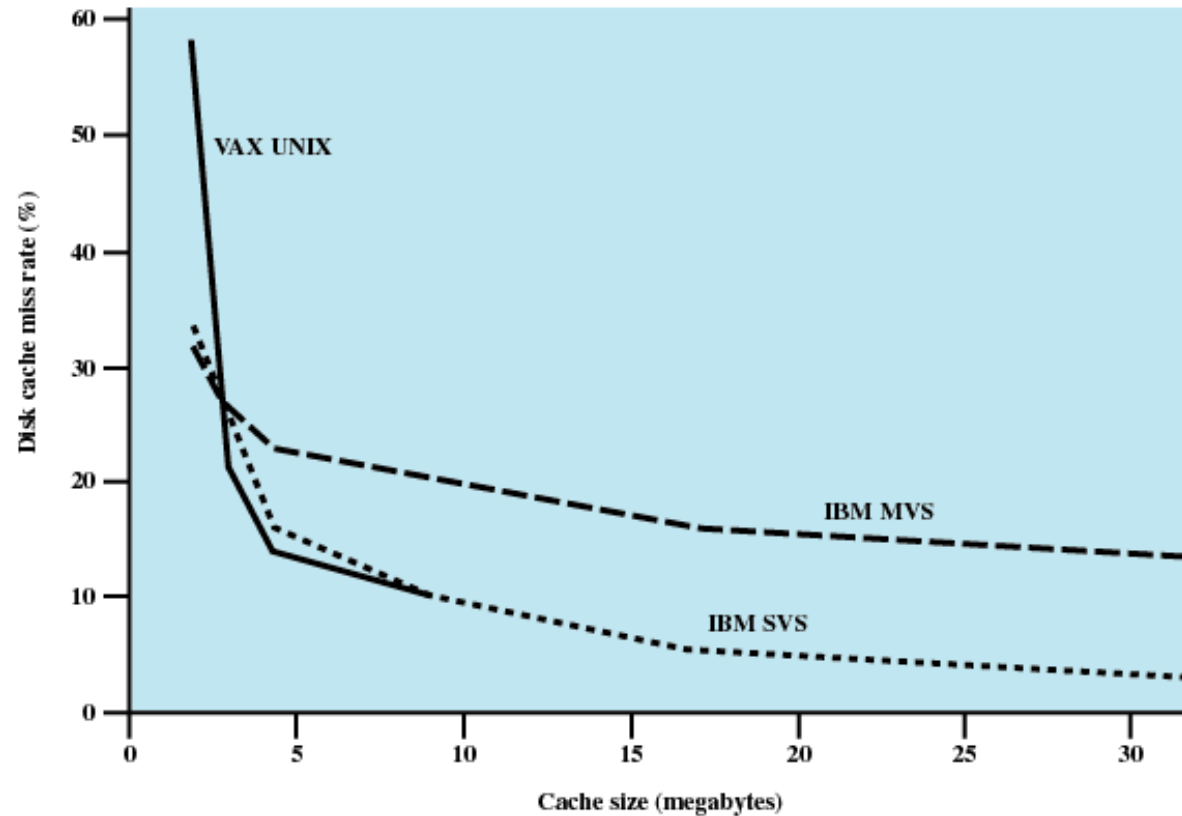


Figure 11.10 Some Disk Cache Performance Results Using LRU

Caching

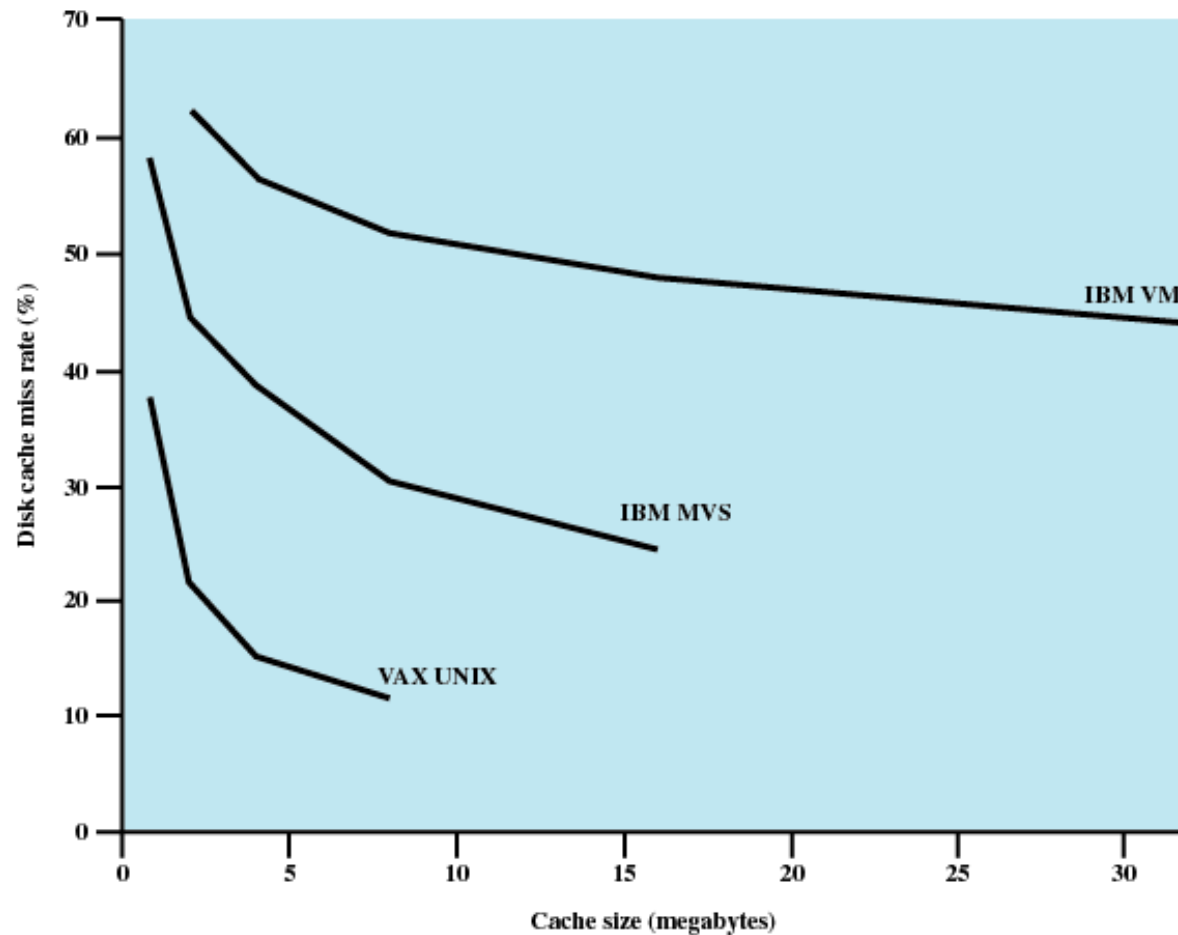
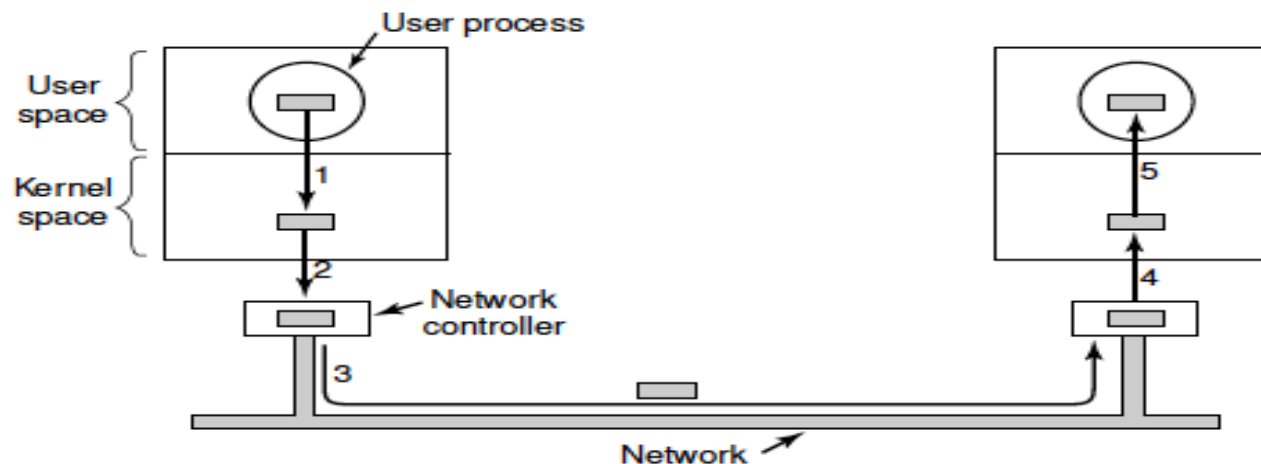


Figure 11.11 Disk Cache Performance Using Frequency-Based Replacement [ROBI90]

Buffering

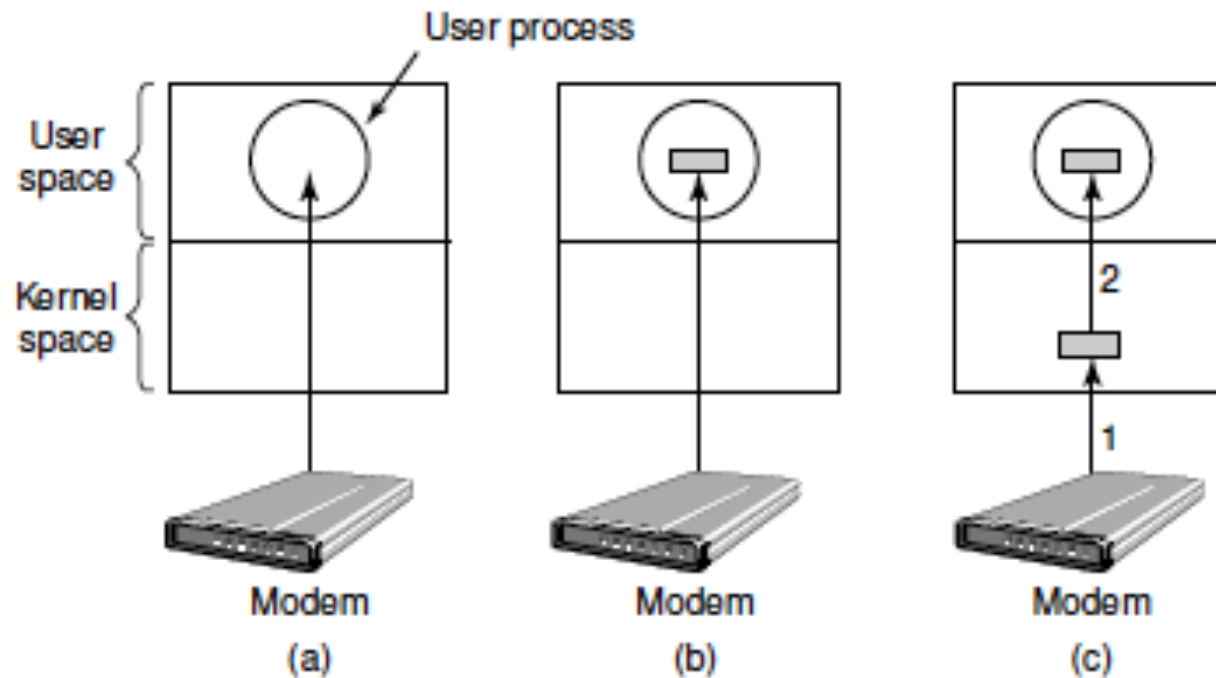
Caching e Buffering sono diversi!!!

- Memorizzazione transitoria dei dati in memoria mentre essi sono trasferiti fra due dispositivi o tra una applicazione ed un dispositivo.
- Permette di disaccoppiare la chiamata di sistema di scrittura con l'istante di effettiva uscita dei dati (output asincrono).
 - Il **produttore** ed il **consumatore** hanno velocità diverse (doppio buffer)
 - I dispositivi trasferiscono dati in blocchi di dimensioni diverse
- Eccessivo uso della bufferizzazione incide sulle prestazioni

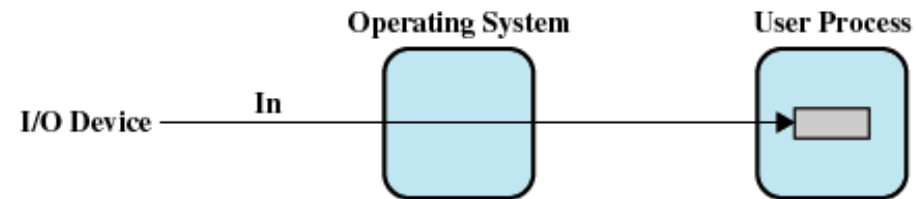


Buffering

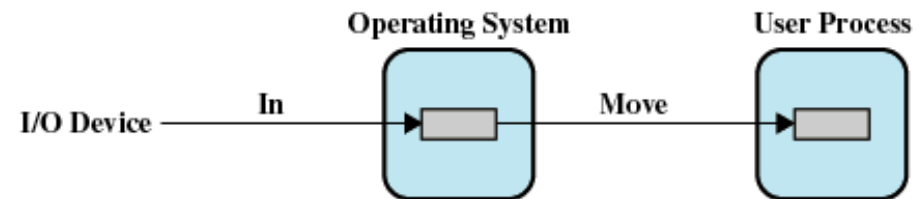
- a) Non bufferizzato: inefficiente
- b) Bufferizzazione in spazio utente: problemi con la memoria virtuale
- c) Bufferizzazione in kernel: bisogna copiare i dati, con blocco dell'I/O nel frattempo.



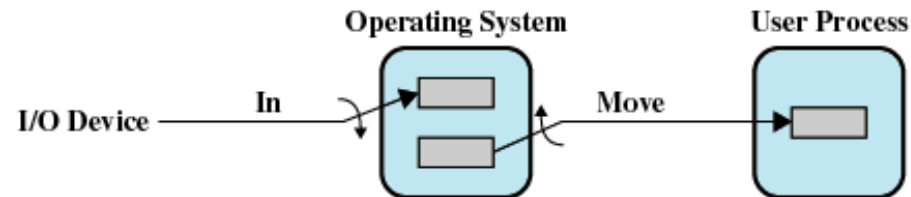
Buffering



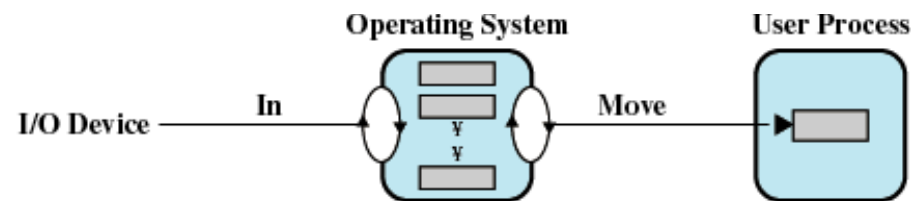
(a) No buffering



(b) Single buffering



(c) Double buffering



(d) Circular buffering



File-System

✓ Interfaccia del File System

- ✓ Tipo di Dato Astratto File
- ✓ File Ordinari
- ✓ Directory
- ✓ File Speciali
- ✓ Protezioni

✓ Realizzazione del File System

- ✓ Struttura dei file system
- ✓ Realizzazione delle directory
- ✓ Realizzazione dei file (assegnazione dei blocchi liberi)
- ✓ Cache (dei blocchi e delle pagine) e Buffering

■ File System di Linux

- Virtual File System
- Ext2 File System
- Ext3 File System
- Proc File System
- Network File System



File System di Linux

■ Interfaccia del File System

- Il file system di linux appare adotta una interfaccia a grafo con la stessa semantica degli altri sistemi Unix.

■ Realizzazione del File System

- Il file system di linux deve essere in grado di gestire diversi tipi di file system contemporaneamente.
- I dettagli implementativi di ogni tipo di file system vengono nascosti da uno strato comune a tutti, il **file system virtuale (VFS)**.
- VFS è stato progettato secondo i principi della programmazione ad oggetti.
- I tre tipi di oggetti principali definiti dal VFS sono:
 - > *inode-object* e *file-object* rappresentano file singoli
 - > *file-system-object* rappresenta un intero file system



File System di Linux

Virtual File System

■ Media based

- ext2 - Linux native
- minix - Minix
- ufs – BSD Unix
- sysv – System V Unix
- fat - DOS FS
- vfat - win 95
- NTFS - NT 's FS
- hpfs - OS/2
- Isofs - CDRom
- hfs - Macintosh
- affs - Amiga Fast FS
- adfs - Acorn-strongarm

■ Network

- nfs
- Coda
- AFS - Andrew FS
- smbfs – Lan Manager
- ncpfs - Novell

■ Journaling

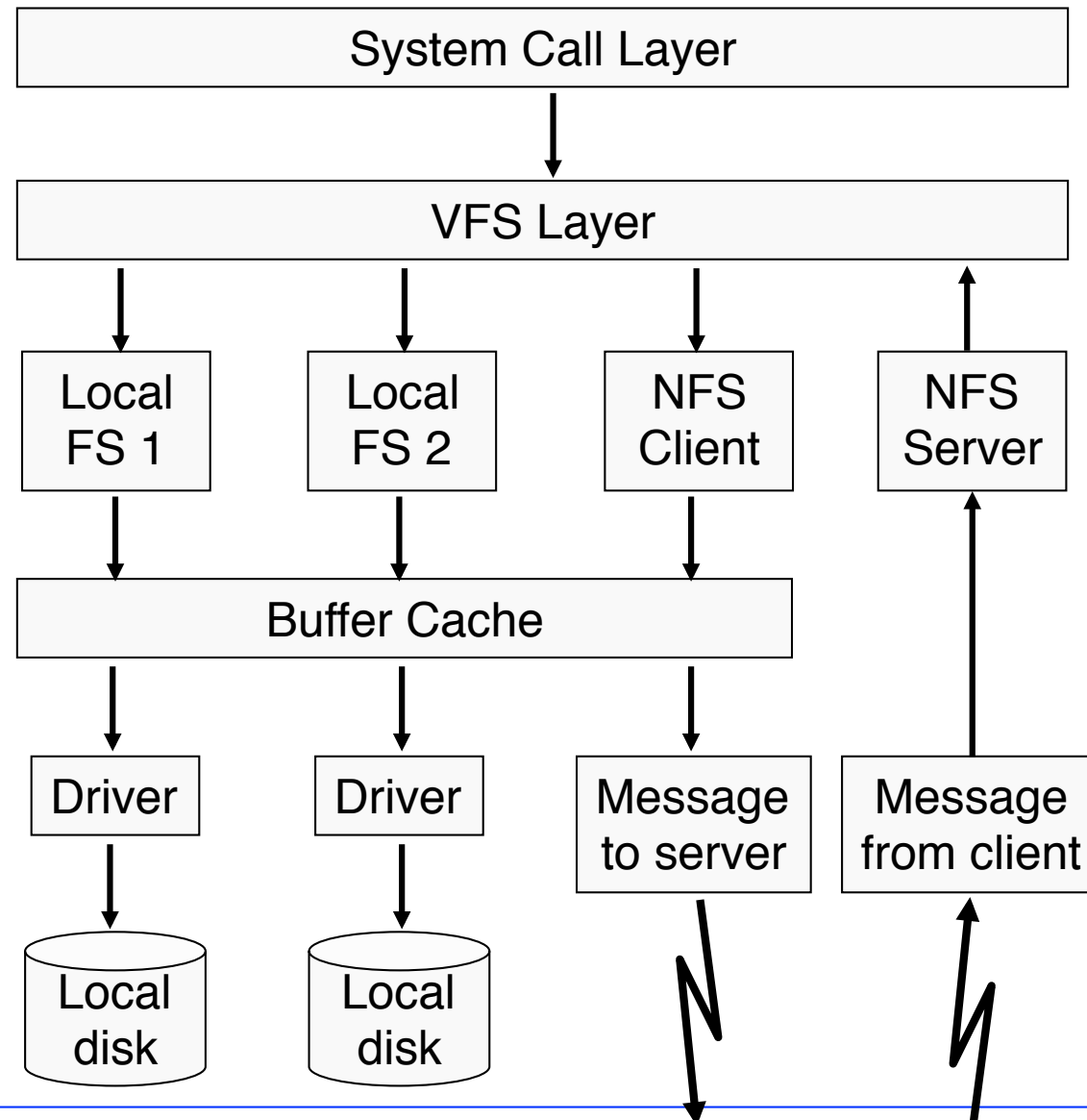
- ext3 - Linux native
- ext4 - Linux native
- reiserfs - Linux native

■ Special ones

- procfs - proc
- umsdos - Unix in DOS
- userfs - redirector to user

File System di Linux

Virtual File System





File System di Linux

Virtual File System

■ Layout di un file system

Boot Block	Superblock	Gestione spazio libero	Gestione spazio occupato	Root dir	File e Directory
------------	------------	------------------------	--------------------------	----------	------------------

■ Cosa contiene un superblocco in Linux?

- meta-informazioni sull'intero file system
- qual'è il file system contenuto nella partizione
- quanti blocchi esistono, quanti sono liberi
- qual' è la dimensione dei blocchi
- informazioni specifiche dei particolari file system



File System di Linux

Ext2 File System

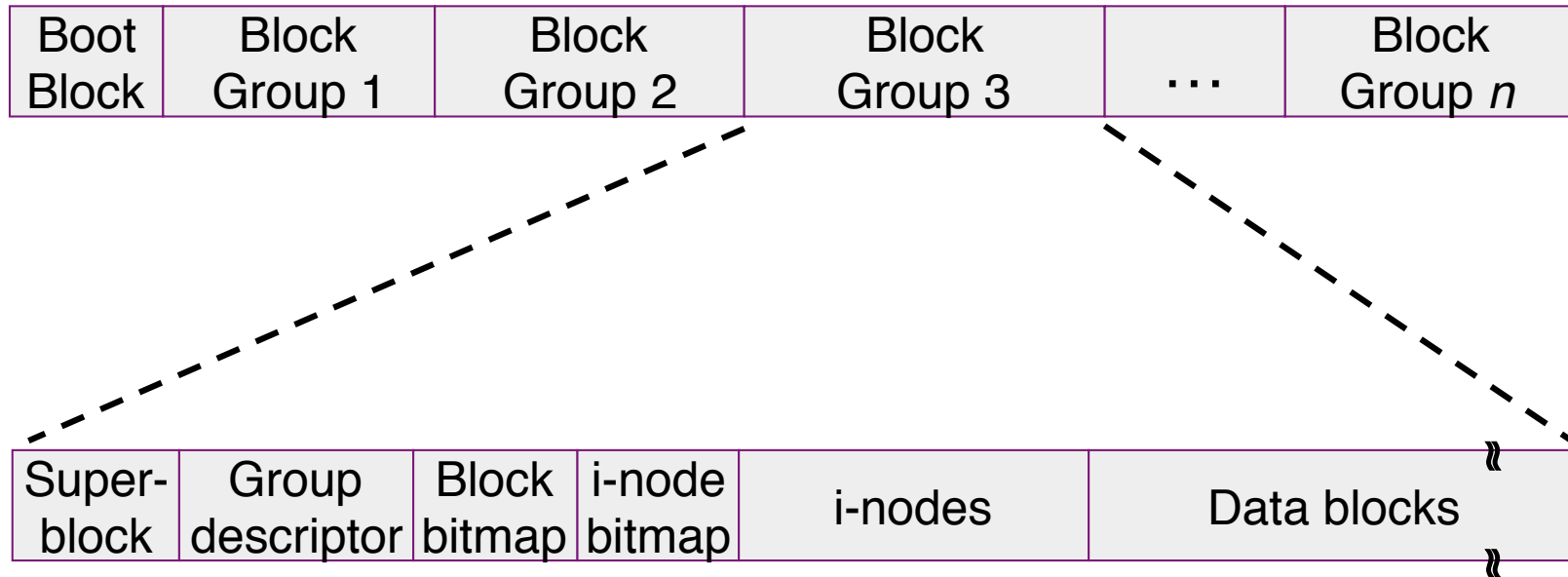
- Ext2fs utilizza un meccanismo di allocazione dei blocchi di un file simile a quello di BSD Fast File System (ffs).
- Le principali differenze sono:
 - In ffs, i blocchi sono da 8Kb e sono suddivisi in frammenti da 1Kb per ridurre la frammentazione interna nell'ultimo blocco.
 - Ext2fs non usa frammenti, esistono solo blocchi da **1Kb**, anche se sono permessi anche blocchi da 2Kb e 4Kb.
 - Ext2fs utilizza politiche di allocazione progettate in modo tale che una richiesta di I/O riguardante diversi blocchi può essere effettuata con una sola operazione:
 - > suddivide un file system in **gruppi di blocchi** e tenta di assegnare ad un file solo blocchi appartenenti ad un unico gruppo;
 - > tenta di posizionare blocchi logicamente adiacenti in blocchi fisicamente adiacenti sul disco.



File System di Linux

Ext2 File System

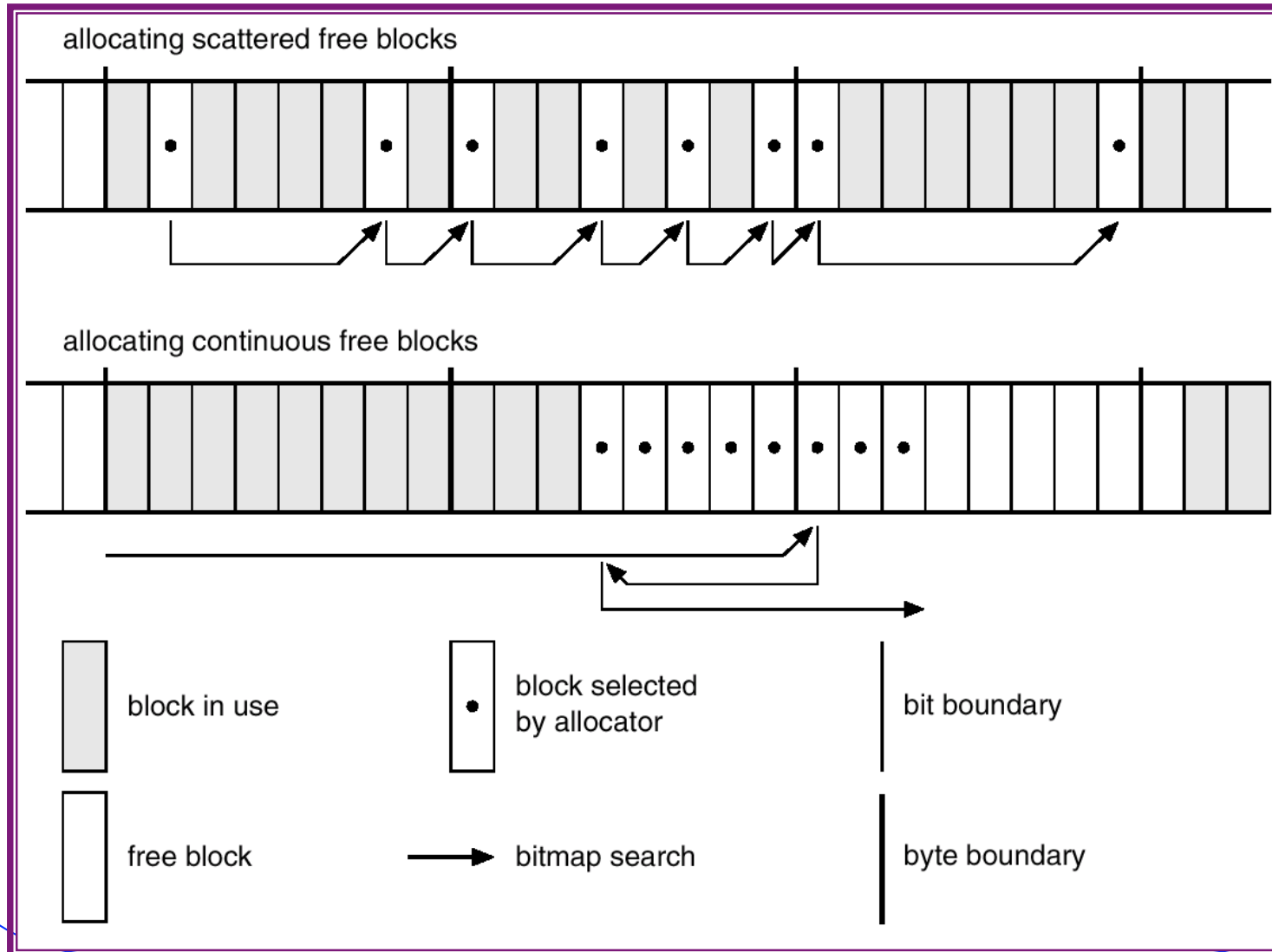
Gruppo di blocchi



File System di Linux

Ext2 File System

Politica di assegnazione dei blocchi





File System di Linux

Ext3/Ext4 File System

- Ext3/Ext4 – sono versioni *journaling* di Ext2 (ovvero con annotazione delle modifiche)
 - Largamente basato su codice Ext2
 - Inserisce un *journal* (.journal file) in Ext2
 - > Usato per esempio da Android 2.3 in poi
- Come funziona il *journal* ?
 - Ogni operazione sul file system viene fatta in due passi:
 - > una copia dei blocchi da scrivere viene scritta nel journal prima del *commit*
 - > dopo il *commit*, i blocchi vengono scritti nelle effettive destinazioni
 - Al momento del recovery (distinguiamo due casi):
 - > prima del *commit*: rimozione → fs inalterato
 - > dopo il *commit*: replica delle operazioni



File System di Linux

Ext3 File System

■ Tre modalità di funzionamento:

- Journal → rischio basso
 - > Nel journal finiscono sia i metadati che i dati (vedi slide precedente)
 - > Molto costoso
- Ordered → rischio medio
 - > Prima vengono fatte le operazioni sui dati
 - > Poi i metadati vengono scritti sul journal
 - > Poi i metadati sono scritti nella destinazione effettiva
 - > Default
- Writeback → rischio alto
 - > Nel journal finiscono solo i metadati
 - > Più efficiente



File System di Linux

Proc File System

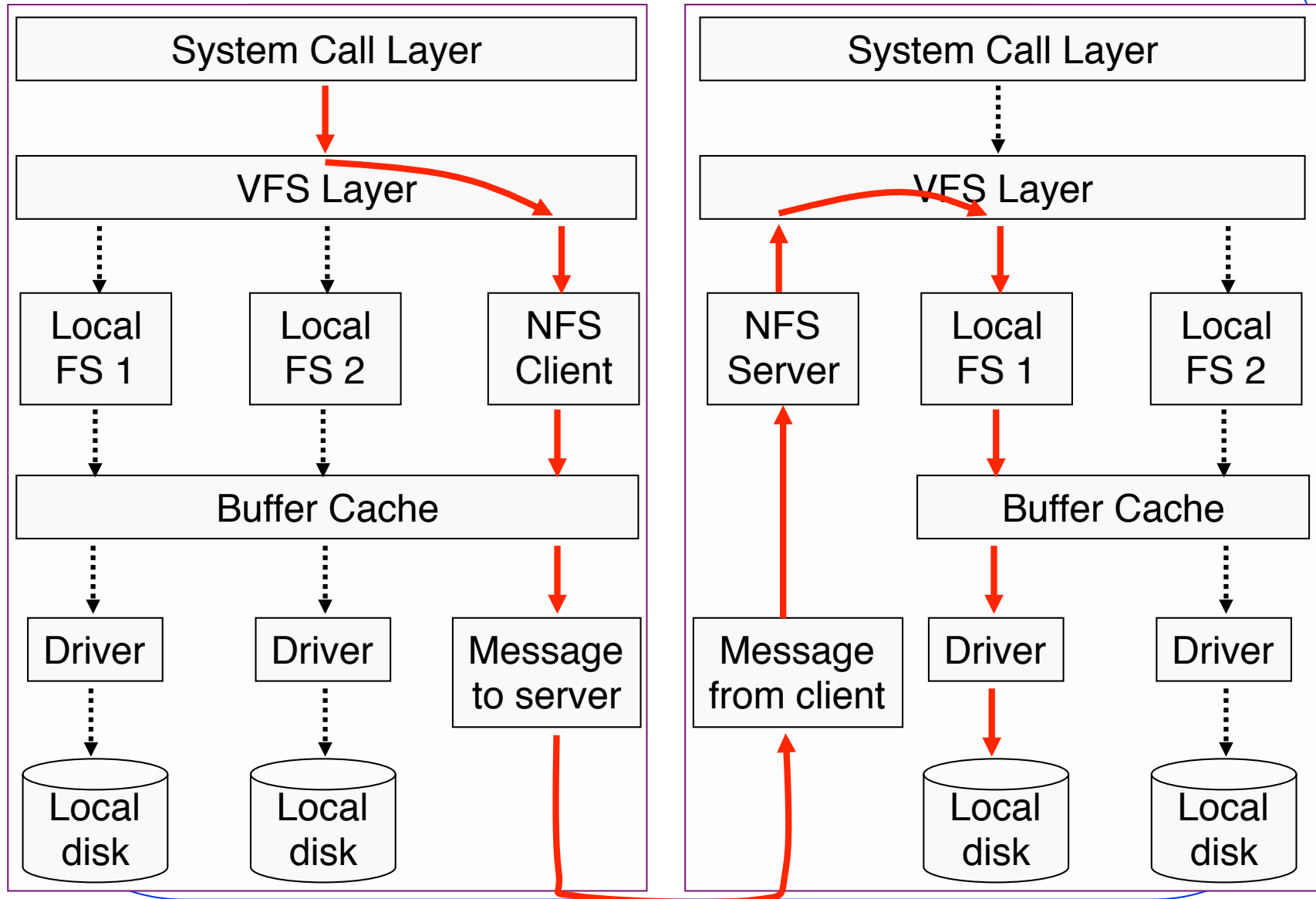
- Il file system *proc* non serve per archiviare in modo permanente dati, ma serve da interfaccia per altri servizi, quindi:
 - non memorizza dati,
 - il suo contenuto è calcolato "on demand" a seconda delle richieste dei processi utenti
- *proc* deve implementare una struttura di directory e il contenuto dei file; questo significa anche definire un *inode* number univoco e persistente per ogni directory e file contenuto nel file system
 - il numero di *inode* identifica l'operazione richiesta da un utente che cerchi di leggere un certo file o di esaminare una certa directory
 - quando i dati vengono letti, *proc* raccoglie le informazioni appropriate, le formatta in forma testuale e le copia nel buffer di lettura del processo richiedente
- **Esempio:** il comando `ps`
 - Invece di essere un processo privilegiato con accesso alla descrizione dei processi direttamente in memoria virtuale
 - È un processo ordinario che legge i dati contenuti nel file system *proc*

File System di Linux

Network File System

Client kernel

Server kernel

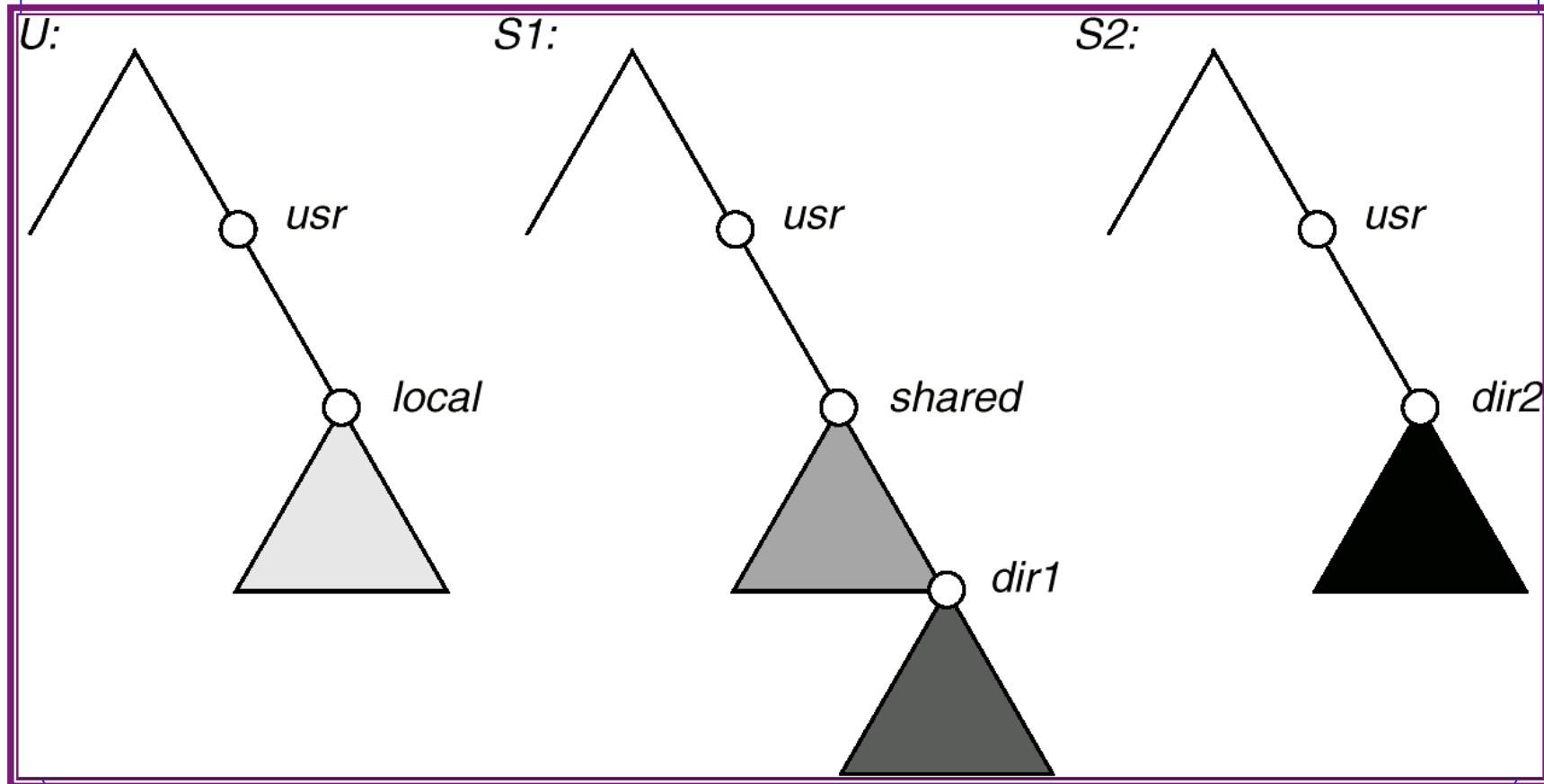


File System di Linux

Network File System

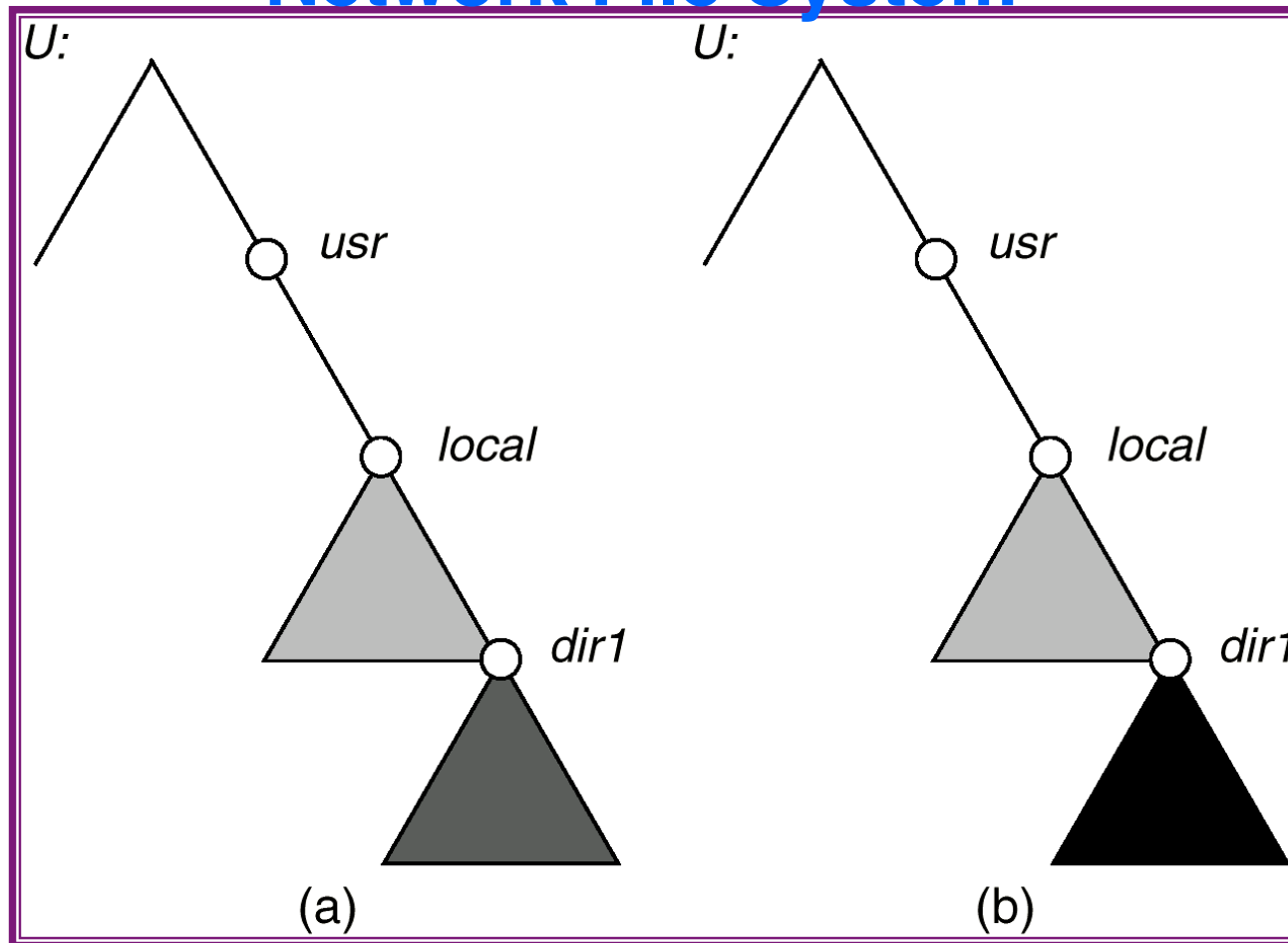
Esempio

Dati tre file system indipendenti



File System di Linux

Network File System



Montaggio di
S1 : /usr/shared in
U : /usr/local

Montaggio in cascata di
S2 : /usr/dir2 in
U : /usr/local/dir1
ottenuto al passo precedente